$$s_i + t_{ij} - M(1 - x_{ij}) \leqslant s_j, \quad \forall i, j \in N, i \neq j \tag{15.19}$$

$$a_i \leqslant s_i \leqslant b_i, \quad \forall i \in N \tag{15.20}$$

$$x_{ij} \in \{0,1\}, \quad \forall i, j \in N, i \neq j \tag{15.21}$$

15.2.2 分支定价算法完整流程及伪代码

前面章节简单回顾了 VRPTW 的列生成细节。下面详细介绍分支定价算法的具体流程和相应的伪代码。

显然，当 RLMP 的解中所有 x_r 取值均为整数时，该解才是原问题的可行解。根据前文中的描述，我们用分支定价算法来求解该问题。分支定价算法的主要思路如下：

（1）首先为 RMP 生成一些初始的列。并构建初始 RMP(15.5)；

（2）求解 RMP 的松弛问题 (15.10)，得到对偶变量；

（3）根据对偶变量更新子问题目标函数，求解子问题，得到一些检验数为负的路径，将其加入 RMP 中；

（4）循环步骤（2）和（3），直到没有新的检验数为负的路径生成为止；

（5）求解最终的 RMP 的线性松弛 (15.10), 如果得到了整数解，则算法结束，返回最优解；如果解不是整数解，则进行分支，创建子节点以及对应的子节点的模型，更新上界和下界，并在每个节点循环步骤（2）和（3）。直到 BB tree 中的叶子节点集合为空，或者上界和下界相等，算法终止。

注意： 在创建分支子节点时，首先需要执行分支约束对应的操作（比如说将分支的列从 RMP 中删除），并在该节点的子问题中加入相应的分支约束。然后，可以保留该子节点的 RMP 中剩余的列，在此基础上，继续执行列生成算法，生成新的列，并加入该子节点的 RMP 中。

图 15.1 是分支定价算法的程序框图。

注意： 上面框图中，在分支中做的事情比较多。主要分为下面的几部分。

（1）根据分支策略，创建子节点。

（2）在每个子节点中，添加对应的分支约束。以分左支为例（分支变量不能再出现）。我们首先需要将分支变量涉及的列从 RMP 中删除，更新 RMP，并将更新后的 RMP 作为该节点的 RMP（有时候，由于问题本身的特性，还需要在 RMP 中添加其他相应的约束）。然后，还需要在子问题中也添加由于分支产生的相关约束，并将更新后的子问题作为该节点的子问题。

（3）执行完上述操作后，还需要将加入了分支约束且删除了分支列的 RMP 和加入了分支约束的 Subproblem 的两个新的节点，加入 BB tree 的队列中。

由于假设所有车辆都是同质的，因此所有车辆的子问题都是相同的，所以可以认为只有一个子问题。下面给出分支定价算法的伪代码。

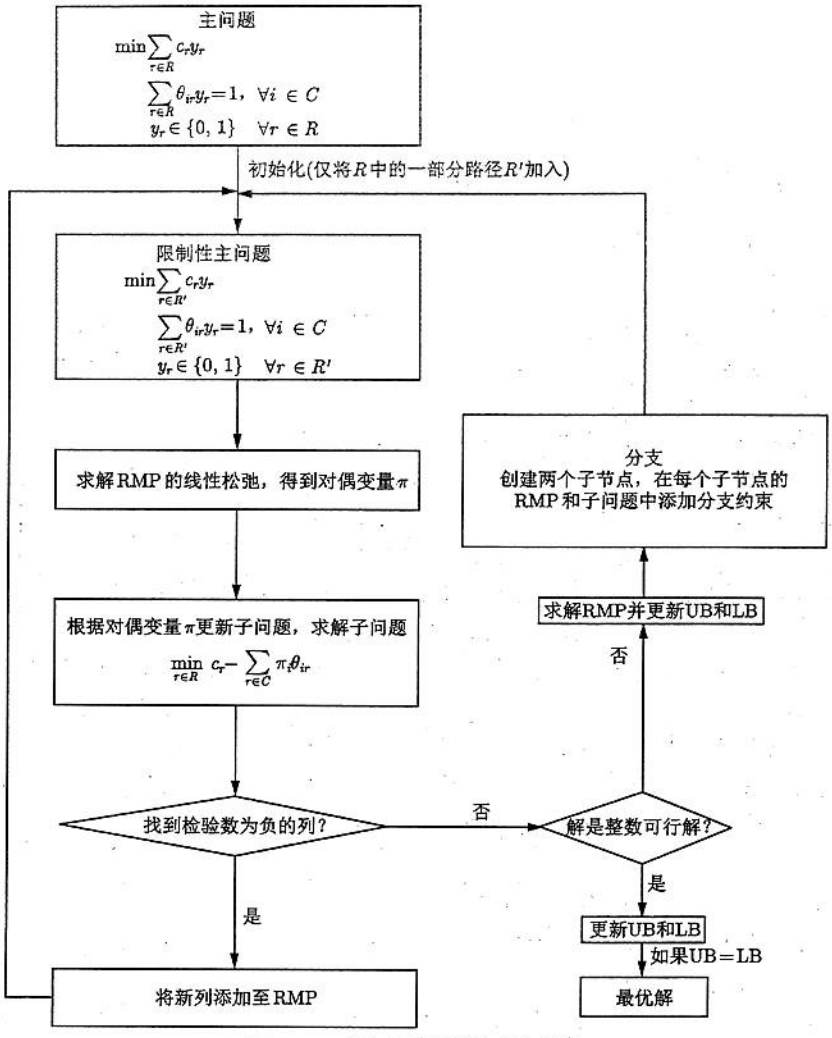


图 15.1 分支定价算法的程序框图

下面，我们给出分支定价算法的伪代码。

Algorithm 17 分支定价算法

1: 初始化: 生成初始列 A (例如用启发式), 构建 RMP_0
2: 设置 $\epsilon \leftarrow$ 一个小的负容差 (如, -0.001)
3: 设置 BB tree 的节点集合 $Q \leftarrow \varnothing$
4: $UB \leftarrow \infty$
5: $LB \leftarrow -\infty(\text{or } 0)$

```
当前最优解incumbent ← Null
对偶变量 π ← 求解 RMP_0 的线性松弛, 即, RLMP_0
求解 Subproblem(s) SP_0 (已包含对偶变量 π)
创建 BB tree 的根节点 N_0 ← {RMP_0, SP_0}
将根节点加入 BB tree 的节点集合中 Q ← Q ∪ {N_0}
while Q 非空或 UB−LB > |ε| do
    P ← Q.pop()                          /* 深度优先 */
    RMP ← P.RMP
    SP← P.SP
    σ ← 求解 SP 并得到目标值
    while σ < ε do                       /* 列生成算法开始 */
        得到新列 (可多列) 并加入 RMP 中
        对偶变量 π ← 求解 RMP
        用对偶变量 π 更新 SP
        σ ← 求解 SP(π) 并得到目标值
    end while                            /* 列生成算法结束 */
    y* ← 求解最终 RMP 的线性松弛 (RLMP) 并得到解
    if y* 是整数可行解 then
        if UB > y* 对应的目标函数 then
            UB ← y* 对应的目标函数
            incumbent ← y*
        end if
        点 P 被剪枝 (根据最优性剪枝)
    else if y* 为空 then
        点 P 被剪枝 (根据非可行性剪枝)
    else if y* 是小数解 then
        if UB < y* 对应的目标函数 then
            点 P 被剪枝 (根据界限剪枝)
        else
            ( **** Branch Scheme **** )
            根据分支策略创建子节点, 其集合为 N
            for 每个子节点 i ∈ N do
                更新 RMP_i (加入分支约束)
                更新 SP_i (加入分支约束导致的相关约束)
                N_i ← {RMP_i, SP_i}
            end for
```

```
        end if
        Q ← Q ∪ N
        LB_temp ← 所有叶子节点的最小局部下界 min_{i∈Q} { ( z*_RLMP + Σ_{k=1}^{K} σ̃*_k )_i }
        if LB_temp > LB then
            更新 LB ← LB_temp
        end if
    end if
end while
return incumbent
```

上述伪代码中，incumbent就是当前最优解。我们通过优先队列来存储 BB tree 的节点，并且每次pop()都弹出优先队列队首元素，并将其从队列中删除。

15.2.3 分支策略

前面的章节中我们详细描述了分支定价算法的具体流程。但是在分支策略的部分并没有进行详细的阐述。本节具体介绍几种典型的分支策略。分支定价算法中的分支操作和一般的分支定界算法中的分支策略还是有一些不同的。在分支定价算法中，主问题的变量和子问题是有一定关联的。我们求解 RLMP，得到了小数解，如果针对 RLMP 中的变量进行分支，要想在子节点中被禁止的列不再出现，那就需要在子问题中也加入相应的分支约束。因此，分支定价算法中的分支，是牵一发而动全身的。

在 VRPTW 中 BB tree 的一个节点处，用列生成算法生成了所有检验数为负的列，然后求解最终的 RMP 的线性松弛，得到了最优解 y^*。如果 y^* 是整数，则不用分支；如果 y^* 是小数，我们就需要针对 y^* 进行分支。我们令

$$r_0 = \{i_0 = \text{depot} \to i_1 \to i_2 \to \cdots \to i_{p+1} = \text{depot}\}$$

是当前 RMP 的所有决策变量取值中分数部分最不可行的决策变量对应的路径（也可以是取值最接近 0.5 的路径）。假定 y_{r_0} 的取值为 $\bar{y}_{r_0}$，我们就取变量 y_{r_0} 为分支变量。下面我们来介绍 2 种分支策略。

1. 基于路径的分支

前文中已经介绍了一种很直观的分支方法，即在左分支中禁止路径 r_0 被生成，右分支中一定要生成 r_0。对于左分支，只需要加入一个约束即可，即

$$\sum_{e \in r_0} x_e \leqslant |S_{r_0}| - 1$$

其中，$|S_{r_0}|$ 为路径 r_0 中包含的顾客点的个数，并且要将 r_0 对应的列以及 y_{r_0} 从左分支的 RMP 中删去。

对于右分支，为了保证 r_0 一定被生成，我们可以将 r_0 保留在右分支的 RMP 中，并且在右分支的子问题中，删去 S_{r_0} 中的所有顾客点。

在前文提到，这种分支方法非常不好，会导致左右子树非常不平衡。这点不难理解，我们在左分支中仅加入了一条约束，但是在右分支中却删去了若干顾客点，这导致左分支的模型仅仅发生了微小的变化，而右分支的问题规模明显减小，求解难度明显减小，从而导致左右子树极不平衡。正是由于这个原因，基于路径的分支一般不被采用。

2. 基于弧段的分支

基于弧段的分支是实际中常用的一种分支策略，该分支策略下，左右子树相对较为平衡。接下来我们来详细地介绍该分支策略（Desaulniers et al., 2006）。

由于 $0 < \bar{y}_{r_0} < 1$，且 RMP 的约束右端项全部为 1，因此在 RMP 的其他所有列中，一定至少存在一列，该列对应的路径中至少包含了路径 r_0 中一个客户点。即，RMP 中一定有至少一条其他路径至少包含了客户集合 $S(r_0) = \{i_1, i_2, \cdots, i_p\}$ 中的一个客户点。

接下来以一个简单的例子来介绍分支定价算法求解 VRPTW 时，是如何分支的。假设我们从 RMP 中抽取出 4 列，且这 4 列对应的路径为

$$\begin{aligned}
\text{route}_1 &= \{0-3-2-5-6-0\} \\
\text{route}_2 &= \{0-1-2-4-6-0\} \\
\text{route}_3 &= \{0-1-3-5-4-0\} \\
\text{route}_4 &= \{0-1-6-5-3-0\}
\end{aligned}$$

它们对应的决策变量 y^* 的取值为

$$(\boldsymbol{y}^*)^{\mathrm{T}} = [0,\ 0.6,\ 0.4,\ 0]$$

依照上面的描述，选择的分支变量即为 $y_2 = 0.6$。相应地，y_2 对应的路径为

$$\text{route}_2 = \{0-1-2-4-6-0\}$$

下面写出 RMP 中跟这 4 列相关的部分，即

$$\begin{array}{lccccccccccc}
\min & c_1y_1 & + & c_2y_2 & + & c_3y_3 & + & c_4y_4 & \cdots & & \\
\text{s.t.} & & & y_2 & + & y_3 & + & y_4 & \cdots = & 1 \\
& y_1 & + & y_2 & + & & & & \cdots = & 1 \\
& y_1 & & & + & y_3 & + & y_4 & \cdots = & 1 \\
& & & y_2 & + & y_3 & & & \cdots = & 1 \\
& y_1 & & & + & y_3 & + & y_4 & \cdots = & 1 \\
& y_1 & + & y_2 & & & + & y_4 & \cdots = & 1 \\
0 \leqslant & y_1, & & y_2, & & y_3, & & y_4, & \cdots \leqslant & 1
\end{array} \tag{15.22}$$

容易观察到，route_2 中访问的所有点的集合为 $\{1,2,4,6\}$，其中客户点 2 除了在 route_2 中，在 route_1 中也被访问到了。

由于 $0 < y_2 = 0.6 < 1$，因此对于除了路径 r_0 以外的路径，一定存在一条路径 r，满足 $y_r > 0$，并且至少与路径 r_0 经过了一个相同的点。因此，一定存在 $q \in \{1,2,\cdots,p\}$，满足路径 r 包含客户 i_q 但是不包含弧 (i_q, i_{q+1}) 或者弧 (i_{q-1}, i_q)。换句话说，就是路径 r_0 与路径 r 不完全相同。

我们已经选出了分支路径 y_{r_0}，具体来讲就是 y_2。接下来穷举 RMP 中所有已经生成的列，选择第一条满足下面条件的路径 r 作为要生成分支弧段的路径：

（1）$\bar{y}_r > 0$；

（2）与路径 r_0 至少有一个重合的顾客点。

然后选择 (i_q, i_{q+1}) 作为分支弧段，其中，$q \in \{1,2,\cdots,p\}$ 是满足 $(i_q, i_{q+1}) \notin r$ 的最小的下标（或者如果有 $(i_q, i_{q+1}) \in r, \forall q \in \{1,2,\cdots,p\}$，可以选择弧 $(i_0 = \text{depot}, i_1)$）。

我们用上面的例子来解释这个过程。我们选取 $r_0 = \text{route}_2 = \{0-1-2-4-6-0\}$ 作为分支路径，另外的 3 条路径中 route_1 也包含 $\{2\}$ 这个客户点。我们发现 route_1 和 route_2 有不重合的点，正好满足上面的所有条件，因此选择 route_1 作为 r。

接下来，子节点将会按照以下规则产生：在第 1 个分支中，弧 (i_q, i_{q+1}) 被禁止访问。在第 2 个分支中，如果在一个解对应的路径中同时访问了点 i_q 和 i_{q+1}，则这两个点只有在被弧 (i_q, i_{q+1}) 连接的情况下，这条路径才能被产生，否则这条路径不能被产生。换句话说，在第 2 个分支中，我们将禁止下列弧被最优解选中：

（1）(i_q, t), where $t \neq i_{q+1}$；

（2）(s, i_{q+1}), where $s \neq i_q$。

或者可以理解为，在该分支的子问题中，我们将上述弧直接删除。

继续上面的例子：

$$r_0 = \text{route}_2 = \{0-1-2-4-6-0\}$$

选择第一个与其至少共享一个节点并且至少有一个点不同的路径，因此选择

$$r = \text{route}_1 = \{0-3-2-5-6-0\}$$

路径 r 和路径 r_0 共享客户点 2，路径 r 包含点 2 但是不包含弧段 $(1,2)$ 和 $(2,4)$，我们可以选择弧段 $(1,2)$ 当作分支弧段（当然也可以选择 $(2,4)$）。

在第 1 个分支中，弧 $(1,2)$ 是被禁止的。因此我们需要对 RMP 和子问题都做相应的操作。首先在第 1 个分支的 RMP_1 中，将列 y_2 及对应的列删去。并且在对应的子问题 Subproblem_1 中，将弧 $(1,2)$ 禁止，也就是在 ESPPRC 问题中添加分支约束如下：

$$x_{12} = 0$$

这样操作以后，在第 1 个分支子节点中，基于 RMP_1 调用列生成生算法，生成新的列，就不会再出现 $\text{route}_2 = \{0-1-2-4-6-0\}$ 这一列了。

在第 2 个分支中，必须要保证顾客点 1 和顾客点 2 只有被连续访问（也就是弧 $(1,2)$ 被选中时）时，顾客点 1 和顾客点 2 才能被同时加入子问题的最优解对应的路径当中，否则顾客点 1 和顾客点 2 就不能被同时访问到。例如，如果访问顺序是 $1 \to 5 \to 2$，顾客点 1 和顾客点 2 在同一路径中被访问了，但是顾客点 1 的后序节点不是顾客点 2，那么这条路径就不能被 Subproblem_2 生成，并被添加到 RMP_2 中。此外，我们需要将第 2 个分支的 RMP 中用到弧 $(1,2)$ 的其他列全部删除。由于上述例子中不存在这样的列，该步操作可略过。继续后面的分支操作，在 Subproblem_2 中，我们必须禁止满足以下两种情况的弧段被选中：

（1）进入顾客点 2，但是不是从顾客点 1 进入的；

（2）从顾客点 1 出去，但是不是从顾客点 1 到顾客点 2 的。

因此，在这个分支子节点处的 RMP_2 中，我们要保留列 $\text{route}_2 = \{0-1-2-4-6-0\}$，也就是保留决策变量 y_2。并且在分支节点对应的子问题 Subproblem_2 中，我们需要添加一系列的分支约束，即

$$x_{i2} = 0, \qquad \forall (i,2) \in E, i \neq 1$$

$$x_{1j} = 0, \qquad \forall (1,j) \in E, j \neq 2$$

这样就能保证在第 2 个分支节点中，只要顾客点 1 或者 2 被添加到了一个新列中，那么在这个新列中，顾客点 1 和 2 一定是被连续访问的，并且是先访问顾客点 1，再访问顾客点 2。当然，上述操作也可以通过直接从底层网络中删除相关弧实现。

这个分支策略是非常实用的，因为该策略很好地利用了主问题和子问题之间的联系。通过上述介绍可以看到，在分支定价算法中，分支的步骤是同时涉及主问题和子问题的，在分支的时候，需要根据相应的分支规则，同时对节点处的 RMP 和子问题都做处理。

上面这种基于弧段的分支略微有些烦琐，接下来我们介绍另一种更易理解的基于弧段的分支策略（Feillet，2010）。与前文介绍的分支策略不同，我们令左分支中的弧 (i_q, i_{q+1}) 被禁止访问，右分支中的弧 (i_q, i_{q+1}) 被强制访问。为了方便描述，我们首先引入下面的参数：

（1）a_{ij}^r：如果路径 r 中包含弧 (i,j)，则 $a_{ij}^r = 1$，否则 $a_{ij}^r = 0$；

（2）$\bar{f}_{ij}$：$\bar{f}_{ij} = \sum_{r \in R'} a_{ij}^r \bar{y}_r$，表示弧 (i,j) 在当前节点的 RMP 线性松弛问题的最优解中被选中的次数，或者弧 (i,j) 上的流量（有可能是小数），显然 $0 \leqslant f_{ij} \leqslant 1, \forall (i,j) \in E$。

选取分支弧段的方法也略有不同。这里我们选取 $\bar{f}_{ij}$ 最接近 0.5 的弧段作为分支弧段，并令该弧段为 (i_q, i_{q+1})。基于此，令左子节点中 $\bar{f}_{i_q, i_{q+1}} = 0$，即在左子节点的 RMP 中加入分支约束

$$\sum_{r \in R'} a_{i_q, i_{q+1}}^r y_r = 0$$

相应地，令右子节点中 $\bar{f}_{i_q, i_{q+1}} = 1$，即在右子节点的 RMP 中加入分支约束

$$\sum_{r \in R'} a_{i_q, i_{q+1}}^r y_r = 1$$

这种分支约束更易于理解和实现，也更简洁，仅在左右分支节点中各添加了一条约束，并且子问题不用做任何改动。相比第一种基于弧段的分支策略而言，第二种分支策略中左右子树更加平衡。因此，在实际中更建议使用第二种基于弧段的分支策略。

实际上，在第二种基于弧段的分支策略下，子问题也可以做相应的改动，以加快求解。即，在左子节点的子问题中，删去弧 (i_q, i_{q+1})。在右子节点的子问题中，删去弧 (i_q, t), where $t \neq i_{q+1}$ 和 (s, i_{q+1}), where $s \neq i_q$。

15.2.4 界限更新

在分支定界的过程中，采用最基本的界限 (Bounds) 更新，具体来讲，在每步的迭代中，令

$$\mathrm{UB} = z^*_{\mathrm{RMP}} \tag{15.23}$$

$$\mathrm{LB} = z^*_{\mathrm{RLMP}} + \sum_{k=1}^{K} \tilde{\sigma}^*_k \tag{15.24}$$

其中，z^*_{RMP} 为 RMP 的整数可行解对应的目标值，z^*_{RLMP} 为 RMP 的线性松弛问题 RLMP 的最优解对应的目标值，$\tilde{\sigma}^*_k$ 为子问题 k 的最优目标值，即子问题 k 的检验数。

因为篇幅所限，不在此处给出分支定价算法求解 VRPTW 的完整代码。在附配的资源中提供了相关开源代码的链接，请感兴趣的读者自行查阅。

第16章 Dantzig-Wolfe分解算法

Dantzig-Wolfe 分解算法（Dantzig-Wolfe Decomposition Algorithm）由 George B. Dantzig 和 Philip Wolfe 于 1960 年首次提出，并用于求解大规模线性规划（Dantzig and Wolfe，1960）。Dantzig-Wolfe 分解算法是一种通过将大规模问题分解成若干较小规模的问题，从而提升求解效率的方法。通常，该算法都是与列生成算法结合使用的。

16.1 引 例

在有些线性规划中，约束和变量可以按照如下方式进行分解：

集合 1 中的约束只涉及在变量集合 1 中的变量；

集合 2 中的约束只涉及在变量集合 2 中的变量；

.....

集合 k 中的约束只涉及在变量集合 k 中的变量；

集合 $k+1$ 中的约束条件可以涉及任何变量。集合 $k+1$ 中的约束条件称为中心约束条件。

下面举个例子进一步理解这一点，该例子参考文献（Winston，2004；Winston，2006）。

某公司在 2 家工厂（工厂 1 和工厂 2）生产 2 种钢材（钢材 1 和钢材 2）。生产钢材需要铁、煤和鼓风炉。每家工厂都有自己的煤矿，工厂 1 每天可以使用 12 吨煤，工厂 2 每天可以使用 15 吨煤。每家工厂都有自己的鼓风炉，由于类型不同所以生产 1 吨钢材需要的相关资源也不同，如表 16.1所示。工厂 1 每天可以使用 10 小时的鼓风炉，工厂 2 每天可以使用 4 小时的鼓风炉。铁矿厂位于 2 家工厂的中间，每天一共可以提供 80 吨铁。

表 16.1 公司生产资源需求表

产品/吨	需要的铁/吨	需要的煤/吨	需要的鼓风炉工时/时
工厂 1 生产的钢材 1	8	3	2
工厂 1 生产的钢材 2	6	1	1
工厂 2 生产的钢材 1	7	3	1
工厂 2 生产的钢材 2	5	2	1

设该公司生产的所有钢材都运送给同一个客户，每吨钢材 1 的售价是 170 美元，每吨钢材 2 的售价是 160 美元。从工厂 1 运输 1 吨钢材需要 80 美元，从工厂 2 运输 1 吨钢

材需要 100 美元。假设运输成本是唯一可变的成本，建立使该公司利润最大的线性规划模型并求解。

首先定义如下变量：

x_1= 工厂 1 每天生产的钢材 1 的吨数；

x_2= 工厂 1 每天生产的钢材 2 的吨数；

x_3= 工厂 2 每天生产的钢材 1 的吨数；

x_4= 工厂 2 每天生产的钢材 2 的吨数。

公司的总收入是 $170(x_1+x_3)+160(x_2+x_4)$，运输成本是 $80(x_1+x_2)+100(x_3+x_4)$。因此，目标函数利润 z 是：

$$\begin{aligned} z &= 170(x_1+x_3)+160(x_2+x_4)-[80(x_1+x_2)+100(x_3+x_4)] \\ &= 90x_1+80x_2+70x_3+60x_4 \end{aligned}$$

公司面临如下 5 个约束条件：

约束 1：工厂 1 每天使用的煤不能超过 12 吨；

约束 2：工厂 1 每天使用的鼓风炉时间不能超过 10 小时；

约束 3：工厂 2 每天使用的煤不能超过 15 吨；

约束 4：工厂 2 每天使用的鼓风炉时间不能超过 4 小时；

约束 5：工厂 1 和工厂 2 每天使用的铁矿石不能超过 80 吨。

上述 5 个约束条件对应如下 5 条约束：

$$\begin{aligned} 3x_1+x_2 &\leqslant 12 \\ 2x_1+x_2 &\leqslant 10 \\ 3x_3+2x_4 &\leqslant 15 \\ x_3+x_4 &\leqslant 4 \\ 8x_1+6x_2+7x_3+5x_4 &\leqslant 80 \end{aligned}$$

再加上变量取值范围约束 $x_i \geqslant 0$，我们得到该问题的线性规划模型如下：

$$\begin{aligned} \max \quad & z = 90x_1+80x_2+70x_3+60x_4 \\ \text{s.t.} \quad & 3x_1+x_2 \leqslant 12 \\ & 2x_1+x_2 \leqslant 10 \\ & 3x_3+2x_4 \leqslant 15 \\ & x_3+x_4 \leqslant 4 \\ & 8x_1+6x_2+7x_3+5x_4 \leqslant 80 \\ & x_1,\ x_2,\ x_3,\ x_4 \geqslant 0 \end{aligned} \tag{16.1}$$

上述模型约束结构可以用图 16.1 表示。

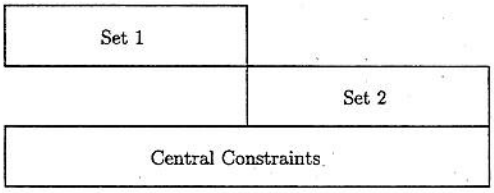


图 16.1 模型约束结构示意图

约束条件集合 1 和变量集合 1 涉及工厂 1 的活动，而不涉及 x_3 和 x_4。约束条件集合 2 和变量集合 2 涉及工厂 2 的活动，而不涉及 x_1 和 x_2。约束条件集合 3 可以看作一个中心约束条件，它和两个变量集合都有耦合。

对于上述问题模型结构，Dantzig 和 Wolfe 开发了 Dantzig-Wolfe 分解算法。利用 Dantzig-Wolfe 分解算法通常可以求解按照这种方式分解的 LP。

分解算法依赖于如下定理的结论。

假设一个 LP 的可行域是有界的，并且这个 LP 可行域的极点（或基本可行解）是 $P_1, P_2, \cdots, P_k$。那么这个 LP 可行域中的任意一个点 x 都可以被表示为 $P_1, P_2, \cdots, P_k$ 的线性组合。也就是说，存在一组权重 $\mu_1, \mu_2, \cdots, \mu_k$ 满足：

$$x = \mu_1 P_1 + \mu_2 P_2 + \cdots + \mu_k P_k \tag{16.2}$$

同时，这组权重必须满足下面的条件：

$$\mu_1 + \mu_2 + \cdots + \mu_k = 1, \quad \forall i = 1, 2, \cdots, k \tag{16.3}$$

$$\mu_i \geqslant 0 \tag{16.4}$$

满足 (16.3) 的权重向量的任意组合称为凸组合。因此该定理表明，如果一个 LP 的可行域是有界的，那么其中的点就可以被表示为该 LP 可行域中极点的凸组合。

为了解释 Dantzig-Wolfe 分解算法的基本思想，假设变量集合被分解成集合 1 和集合 2。

Dantzig-Wolfe 分解算法的步骤如下。

1. 第 1 步

设变量集合 1 中的变量是 $x_1, x_2, \cdots, x_{n_1}$。我们把变量表示成约束条件集合 1（只涉及变量集合 1 中变量的约束条件）的可行域中极点的凸组合。如果设 $P_1, P_2, \cdots, P_k$ 是这个可行域的极点，那么可以把约束条件集合 1 中的可行域中的点

$$\begin{bmatrix} x_1 \\ x_2 \\ .. \\ x_{n_1} \end{bmatrix} \tag{16.5}$$

记作如下形式：

$$\begin{bmatrix} x_1 \\ x_2 \\ .. \\ x_{n_1} \end{bmatrix} = \mu_1 P_1 + \mu_2 P_2 + \cdots + \mu_k P_k \tag{16.6}$$

其中，$\mu_1 + \mu_2 + \cdots + \mu_k = 1$ 且 $\mu_i \geqslant 0, i = 1, 2, \cdots, k$。

2. 第 2 步

把变量集合 2 中的变量 $x_{n_1+1}, x_{n_2+1}, \cdots, x_n$ 表示成约束条件集合 2 的可行域中极点的凸组合。如果设这个可行域中的极点为 $Q_1, Q_2, \cdots, Q_m$，那么可以把约束条件集合 2 的可行域中的点记作

$$\begin{bmatrix} x_{n_1+1} \\ x_{n_2+2} \\ \vdots \\ x_n \end{bmatrix} = \lambda_1 Q_1 + \lambda_2 Q_2 + \cdots + \lambda_m Q_m \tag{16.7}$$

其中，$\lambda_1 + \lambda_2 + \cdots + \lambda_m = 1$ 且 $\lambda_i \geqslant 0 (i = 1, 2, \cdots, m)$。

3. 第 3 步

利用 (16.6) 和 (16.7) 把 LP 的目标函数和约束条件表示为关于 μ_i 和 λ_i 的方程。在增加了约束条件（称作凸性约束条件）$\mu_1 + \mu_2 + \cdots + \mu_k = 1$ 和 $\lambda_1 + \lambda_2 + \cdots + \lambda_m = 1$ 以及符号约束条件 $\mu_i \geqslant 0 (i = 1, 2, \cdots, k)$ 和 $\lambda_i \geqslant 0, i = 1, 2, \cdots, m$ 后，我们可以得到如下通常称作限制主问题的 LP（RMP）：

$$\begin{aligned}
\max(\text{or } \min) \quad & [\text{关于 } \mu_i \text{ 和 } \lambda_i \text{ 的目标函数}] \\
\text{s.t.} \quad & [\text{关于 } \mu_i \text{ 和 } \lambda_i \text{ 的中心约束条件}] \\
& \mu_1 + \mu_2 + \cdots + \mu_k = 1 \\
& \lambda_1 + \lambda_2 + \cdots + \lambda_m = 1 \\
& \mu_i \geqslant 0, \qquad \forall i = 1, 2, \cdots, k \\
& \lambda_i \geqslant 0, \qquad \forall i = 1, 2, \cdots, m
\end{aligned}$$

在大规模线性规划问题中，限制主问题（RMP）可能有很多个变量（对应于每组约束条件中的很多个基本可行解）。一般情况下，我们只需要产生特定的 μ_i 或 λ_i 对应的列，而不需要写出完整的限制主问题。

4. 第 4 步

假设限制主问题（RMP）基本可行解可以通过启发式算法或者求解器比较容易得到。接下来使用前文介绍的列生成方法就可以求解限制主问题 (RMP)。

5. 第 5 步

将第 4 步中求出的 μ_i 或 λ_i 的值代入 (16.6) 和 (16.7)。我们就可以得到 $x_1, x_2, \cdots, x_n$ 的最优值。

我们给出 Dantzig-Wolfe 分解算法的伪代码如下。

Algorithm 18 Dantzig-Wolfe 分解算法

```
初始化: 设置 x_i ← 极点的凸组合
while RMP 的解不是最优 do
    用列生成算法求解 RMP
end while
return x_i
```

16.2 块角模型与 Dantzig-Wolfe 分解

上面我们用了一个比较简单的例子引入了 DW 分解算法的具体步骤。本节我们以一个更通用的角度来解释 DW 分解（参考文献（kalvelagen，2003））。

16.2.1 块角模型

我们考虑下面的线性规划问题：

$$\begin{aligned} \min \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b} \\ & \boldsymbol{x} \in \mathbb{R}^n \end{aligned}$$

其中，约束矩阵 $\boldsymbol{A}$ 具有如下特定结构：

$$\boldsymbol{A}\boldsymbol{x} = \begin{pmatrix} \boldsymbol{B}_0 & \boldsymbol{B}_1 & \boldsymbol{B}_2 & \cdots & \boldsymbol{B}_K \\ & \boldsymbol{A}_1 & & & \\ & & \boldsymbol{A}_2 & & \\ & & & \ddots & \\ & & & & \boldsymbol{A}_K \end{pmatrix} \begin{pmatrix} \boldsymbol{x}_0 \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_K \end{pmatrix} = \begin{pmatrix} \boldsymbol{b}_0 \\ \boldsymbol{b}_1 \\ \boldsymbol{b}_2 \\ \vdots \\ \boldsymbol{b}_K \end{pmatrix}$$

约束

$$\sum_{k=0}^{K} \boldsymbol{B}_k \boldsymbol{x}_k = \boldsymbol{b}_0$$

对应着上述约束矩阵的第 1 行，称作中心约束。

Dantzig-Wolfe 分解算法的思想就是将上述模型进行分解，而不是考虑所有约束一起求解。将问题分解为主问题和若干子问题进行求解，主问题考虑中心约束条件，子问题进行分开求解，从而只有一系列更小规模的问题需要我们求解。

16.2.2 Minkowski 表示定理

考虑线性规划问题的可行域：

$$P=\{x|Ax=b,x\geqslant 0\}$$

如果 P 是有界的，那么可以将点 $x\in P$ 表示成 P 的极点 $x^{(j)}$ 的线性组合形式，如下：

$$\begin{aligned}&x=\sum_j\lambda_j x^{(j)}\\&\sum_j\lambda_j=1\\&\lambda_j\geqslant 0\end{aligned}$$

如果可行域不是有界的情况，那需要引入极射线，如下：

$$\begin{aligned}&x=\sum_j\lambda_j x^{(j)}+\sum_i\mu_i r^{(i)}\\&\sum_j\lambda_j=1\\&\lambda_j\geqslant 0\\&\mu_i\geqslant 0\end{aligned}$$

其中，$r^{(i)}$ 是 P 的极射线。上述表述被称作 Minkowski 表示定理。约束 $\sum_j\lambda_j=1$ 即上文提到的凸约束。

也可以把上述表述写作如下统一的形式：

$$\begin{aligned}&x=\sum_j\lambda_j x^j\\&\sum_j\delta_j\lambda_j=1\\&\lambda_j\geqslant 0\\&\mu_i\geqslant 0\end{aligned}\tag{16.8}$$

其中

$$\delta_j=\begin{cases}1, & x^j\text{ 是一个极点}\\0, & x^j\text{ 是一条极射线}\end{cases}$$

通过上述形式就可以把原问题关于变量 x 的形式转换为关于变量 λ 的形式。实际上，随着变量 λ_j 的数量增多，此模型逐渐变得不能直接求解。

16.2.3 模型分解

下面将模型分解成主问题和 K 个子问题，K 个子问题有如下约束形式：

$$\begin{aligned} \boldsymbol{A}_k\boldsymbol{x}_k &= \boldsymbol{b}_k \\ \boldsymbol{x}_k &\geqslant \boldsymbol{0} \end{aligned}$$

而主问题形式如下：

$$\begin{aligned} \min \quad & \sum_k \boldsymbol{c}_k^{\mathrm{T}}\boldsymbol{x}_k \\ \text{s.t.} \quad & \sum_k \boldsymbol{B}_k\boldsymbol{x}_k = \boldsymbol{b}_0 \\ & \boldsymbol{x}_0 \geqslant \boldsymbol{0} \end{aligned}$$

我们将 (16.8) 代入上述主问题，得到

$$\begin{aligned} \min \quad & \boldsymbol{c}_0^{\mathrm{T}}\boldsymbol{x}_0 + \sum_{k=1}^{K}\sum_{j=1}^{p_k}\left(\boldsymbol{c}_k^{\mathrm{T}}\boldsymbol{x}_k^{(j)}\right)\lambda_{k,j} \\ \text{s.t.} \quad & \boldsymbol{B}_0\boldsymbol{x}_0 + \sum_{k=1}^{K}\sum_{j=1}^{p_k}\left(\boldsymbol{B}_k\boldsymbol{x}_k^{(j)}\right)\lambda_{k,j} = \boldsymbol{b}_0 \\ & \sum_{j=1}^{p_k}\delta_{k,j}\lambda_{k,j} = 1, \forall\ k = 1, 2, \cdots, K \\ & \boldsymbol{x}_0 \geqslant \boldsymbol{0} \\ & \lambda_{k,j} \geqslant 0 \end{aligned}$$

其中，p_k 是第 k 个子问题的极点个数。这仍是一个大规模的线性规划问题。虽然行的数量减少了，但是每个子问题的极点和极射线的数量是很多的，这就导致变量 $\lambda_{k,j}$ 的数量很多。然而一开始很多变量是非基变量，我们就不需要把它们纳入进来。我们的想法是应用列生成的思想，只考虑那些检验数有希望为负的变量。

由此得到限制性主问题（Restricted Master Problem），模型如下：

$$\begin{aligned} \min \quad & \boldsymbol{c}_0^{\mathrm{T}}\boldsymbol{x}_0 + \boldsymbol{c}^{\mathrm{T}}\boldsymbol{\lambda}^{'} \\ \text{s.t.} \quad & \boldsymbol{B}_0\boldsymbol{x}_0 + \boldsymbol{B}\boldsymbol{\lambda}^{'} = \boldsymbol{b}_0 \\ & \Delta\boldsymbol{\lambda}^{'} = 1 \\ & \boldsymbol{x}_0 \geqslant \boldsymbol{0} \\ & \boldsymbol{\lambda}^{'} \geqslant \boldsymbol{0} \end{aligned}$$

缺失的变量定义为 0。限制主问题的列是不固定的，随着列生成，会有新的列加入该模型中。变量 $\lambda_{k,j}$ 的潜力可以由其检验数来衡量。我们把中心约束 $\boldsymbol{B}_0\boldsymbol{x}_0 + \boldsymbol{B}\boldsymbol{\lambda}^{'} = \boldsymbol{b}_0$ 对应的对偶变量记作 $\boldsymbol{\pi}_1$，把凸约束 $\sum_j \delta_{k,j}\lambda_{k,j}^{'} = 1$ 的对偶变量记作 π_2^k，则主问题中变量 $\lambda_{k,j}^{'}$ 的检验数可以写作

$$\sigma_{k,j} = (\boldsymbol{c}_k^{\mathrm{T}}\boldsymbol{x}_k^{(j)}) - \boldsymbol{\pi}^{\mathrm{T}}\begin{pmatrix} B_k\boldsymbol{x}_k^{(j)} \\ \delta_{k,j} \end{pmatrix} = (\boldsymbol{c}_k^{\mathrm{T}} - \boldsymbol{\pi}_1^{\mathrm{T}}B_k)\boldsymbol{x}_k^{(j)} - \pi_2^k\delta_{k,j} \tag{16.9}$$

如果子问题是有界的，那么最有潜力的加入主问题的基本可行解 $\boldsymbol{x}_k$ 可以通过求解下面以检验数为目标函数的模型得到：

$$\begin{aligned} \min_{\boldsymbol{x}_k} \quad & \sigma_k = (\boldsymbol{c}_k^{\mathrm{T}} - \boldsymbol{\pi}_1^{\mathrm{T}}B_k)\boldsymbol{x}_k - \pi_2^k \\ \text{s.t.} \quad & \boldsymbol{A}_k\boldsymbol{x}_k = \boldsymbol{b}_k \\ & \boldsymbol{x}_k \geqslant \boldsymbol{0} \end{aligned}$$

寻找这些检验数的过程通常称作 pricing。如果 $\sigma_k^* \leqslant 0$，就可以将新列 $\lambda_{k,j}$ 加入主问题中，该列的成本系数是 $\boldsymbol{c}_k^{\mathrm{T}}\boldsymbol{x}_k^*$。

DW 分解中主问题和子问题的交互可以参见图 16.2。

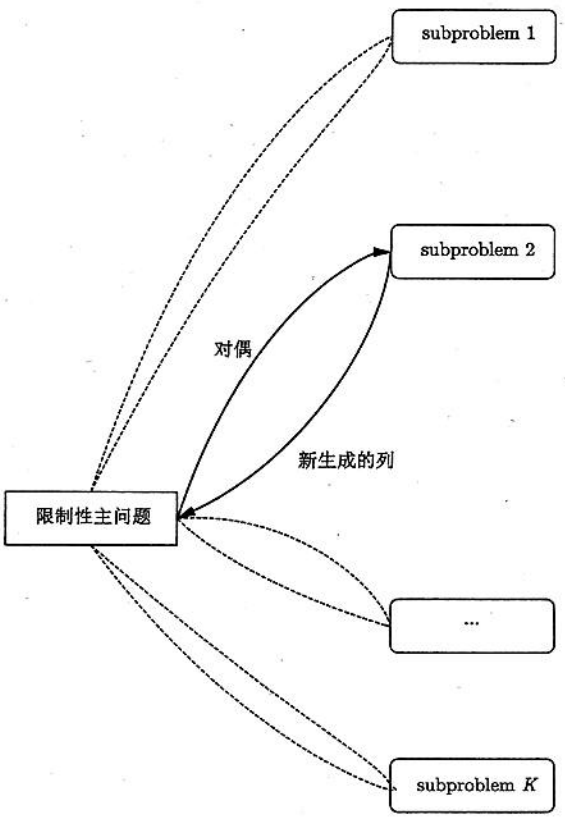


图 16.2 限制性主问题和子问题之间的交互

DW 分解的伪代码如下（kalvelagen，2003）。

Algorithm 19 Dantzig-Wolfe 分解算法

```
{初始化}：选择初始变量的子集
while true do
    {Master problem}
    求解 RMP.
    π_1 ← 中央约束的对偶变量
    π_2^k ← 第 k 个凸组合约束的对偶变量
    {Sub problems}
    for k = 1, ··· , K do
        将 π_1 和 π_2^k 代入 subproblem k
        求解 subproblem k
        if σ_k^* ⩽ 0 then
            将新的候选极点 x_k^* 加入 RMP
        end if
    end for
    if 没有新的候选极点产生 then
        Stop:optimal
    end if
end while
```

关于分解算法的初始化需要根据不同的问题具体实际地分析。在求解子问题过程中，如果子问题不可行，那么原问题即是不可行的。

16.2.4 两阶段法

有时候，一开始的解可能不满足中心约束条件。我们可以通过 Phase I/II Algorithm 来解决（实际上就是单纯形法中的两阶段法）。首先，通过引入人工变量并且最小化它来形成第一阶段问题。关于人工变量的解释本书不再赘述，不熟悉的读者可参考线性规划的相关教材。需要注意的是，第一阶段问题的检验数和第二阶段问题的检验数是有些差别的。

比如说，我们考虑如下中心约束条件：

$$\sum_j x_j \leqslant b$$

加入人工变量 $x_a \geqslant 0$，得

$$\sum_j x_j - x_a \leqslant b$$

这时第一阶段的目标函数是

$$\min \quad x_a$$

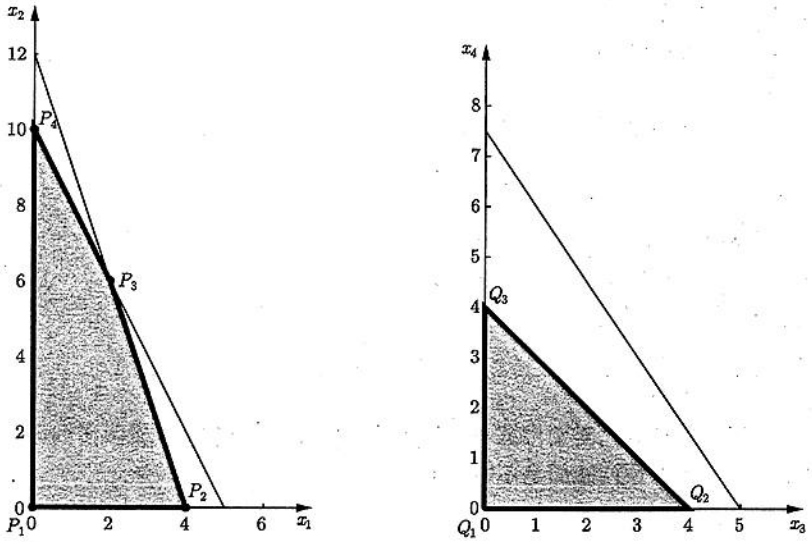
变量 x_j 的检验数如 (16.9) 所示，但是需要 $c_k^{\mathrm{T}}=\mathbf{0}$。

需要提醒的是，在第二阶段开始时需要将人工变量移除。在下面的例子中采取的做法是使该人工变量为 0。

16.3 详细案例

接下来继续求解本章开头提到的例子。

和上文定义的集合 1 和集合 2 一致，可以得到约束条件集合 1 和约束条件集合 2 的可行域，如图 16.3 所示，其中阴影部分是可行域。



(a) 约束条件集合 1 的可行域　　(b) 约束条件集合 2 的可行域

图 16.3　约束条件集合 1 和 2 的可行域

于是，可以把约束条件集合 1 的极点记作

$$P_1=(0,\ 0),\ P_2=(4,\ 0),\ P_3=(2,\ 6),\ P_4=(0,\ 10)$$

把约束条件集合 2 的极点记作

$$Q_1=(0,\ 0),\ Q_2=(4,\ 0),\ Q_3=(0,\ 4)$$

已知变量集合 1=$\{x_1,x_2\}$，则有

$$\text{约束条件集合 1}=\begin{cases}3x_1+x_2\leqslant 12\\ 2x_1+x_2\leqslant 10\end{cases}$$

接下来可以得到约束条件集合 1 的可行点，记作

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \mu_1 \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \mu_2 \begin{bmatrix} 4 \\ 0 \end{bmatrix} + \mu_3 \begin{bmatrix} 2 \\ 6 \end{bmatrix} + \mu_4 \begin{bmatrix} 0 \\ 10 \end{bmatrix} = \begin{bmatrix} 4\mu_2 + 2\mu_3 \\ 6\mu_3 + 10\mu_4 \end{bmatrix} \tag{16.10}$$

其中，$\mu_1 + \mu_2 + \mu_3 + \mu_4 = 1$, $\mu_i \geqslant 0, i = 1,2,3,4$。

变量集合 2=$\{x_3,x_4\}$，则有

$$\text{约束条件集合 } 2 = \begin{cases} 3x_3 + 2x_4 \leqslant 15 \\ x_3 + x_4 \leqslant 4 \end{cases}$$

得到约束条件集合 2 的可行点，记作

$$\begin{bmatrix} x_3 \\ x_4 \end{bmatrix} = \lambda_1 \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \lambda_2 \begin{bmatrix} 4 \\ 0 \end{bmatrix} + \lambda_3 \begin{bmatrix} 0 \\ 4 \end{bmatrix} = \begin{bmatrix} 4\lambda_2 \\ 4\lambda_3 \end{bmatrix} \tag{16.11}$$

其中，$\lambda_1 + \lambda_2 + \lambda_3 = 1$, $\lambda_i \geqslant 0, i = 1,2,3$。

将 (16.10) 和 (16.11) 代入目标函数和中心约束条件中，得到限制主问题。目标函数整理为

$$360\mu_2 + 660\mu_3 + 800\mu_4 + 280\lambda_2 + 240\lambda_3$$

中心约束条件整理为

$$32\mu_2 + 52\mu_3 + 60\mu_4 + 28\lambda_2 + 20\lambda_3 \leqslant 80$$

将该约束条件加入松弛变量 s_1, 得到限制主问题模型如下：

$$\begin{aligned} \max \quad z = \quad & 360\mu_2 + 660\mu_3 + 800\mu_4 + 280\lambda_2 + 240\lambda_3 \\ \text{s.t.} \quad & 32\mu_2 + 52\mu_3 + 60\mu_4 + 28\lambda_2 + 20\lambda_3 + s_1 = 80 \\ & \mu_1 + \mu_2 + \mu_3 + \mu_4 = 1 \\ & \lambda_1 + \lambda_2 + \lambda_3 = 1 \\ & \mu_1, \mu_2, \mu_3, \mu_4, \lambda_1, \lambda_2, \lambda_3, s_1 \geqslant 0 \end{aligned} \tag{16.12}$$

这一步转化模型之后，我们需要解释一下。模型 (16.12) 和模型 (16.1) 已经是完全等价了。直接求解 (16.12) 得到的解，和 (16.1) 的解是等价的。目标函数相同，只是变量不同。但是原始变量可以通过 (16.12) 中的变量转化得到。

接下来使用改进单纯形法和列生成法求解限制主问题。把初始单纯形表称作单纯形表 0。BV(0)=$\{s_1,\mu_1,\lambda_1\}$。

$$\boldsymbol{B}_0 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \boldsymbol{B}_0^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

由于 s_1,μ_1,λ_1 没有出现在限制主问题目标函数中，所以 c_{BV}=[0 0 0], 单纯形表 0 的影子价格是

$$c_{BV}^{T}B_0^{-1}=\begin{bmatrix}0&0&0\end{bmatrix}\begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix}=\begin{bmatrix}0&0&0\end{bmatrix}$$

我们分两个阶段应用列生成的方法。首先确定是否存在与约束条件集合 1 相关且价格有利的 μ_i（由于求解的是最大化问题，第 0 行中系数为负的列是有利的）。与约束条件集合 1 的极点 $\begin{bmatrix}x_1\\x_2\end{bmatrix}$ 相关的系数 μ_i 在限制主问题中具有如下列：

$$\mu_i \text{ 的目标函数系数 } =90x_1+80x_2$$

$$\mu_i\text{在约束条件中的列}=\begin{bmatrix}8x_1+6x_2\\1\\0\end{bmatrix}$$

由以上信息可知，在单纯形表 0 中，对应于 $\begin{bmatrix}x_1\\x_2\end{bmatrix}$ 的 μ_i 的列具有如下价格：

$$c_{BV}^{T}B_0^{-1}\begin{bmatrix}8x_1+6x_2\\1\\0\end{bmatrix}-(90x_1+80x_2)=-90x_1-80x_2$$

由于 $\begin{bmatrix}x_1\\x_2\end{bmatrix}$ 需要满足约束条件集合 1（或工厂 1 的约束条件），所以具有最大负价格的加权 μ_i 是与作为下面 LP 最优解的极点相关的加权。

工厂 1 子问题如下：

$$\begin{aligned}\min\quad & z=-90x_1-80x_2\\ \text{s.t.}\quad & 3x_1+x_2\leqslant 12\\ & 2x_1+x_2\leqslant 10\\ & x_1,x_2\geqslant 0\end{aligned}$$

求解工厂 1 子问题，得解 $z=-800, x_1=0, x_2=10$。这说明，与极点 $\begin{bmatrix}0\\10\end{bmatrix}$ 相关的加权 μ_i 将有最大负价格。由于 $\boldsymbol{P}_4=\begin{bmatrix}0\\10\end{bmatrix}$，这意味着 μ_4 在限制主问题中具有系数 −800。

现在分析和约束条件集合 2 相关的加权，并设法确定具有最大负价格的加权 λ_i。对应于约束条件集合 2 的极点 $\begin{bmatrix}x_3\\x_4\end{bmatrix}$ 的 λ_i 在限制主问题中具有以下列：

$$\lambda_i \text{ 的目标函数系数 } = 70x_3 + 60x_4$$

$$\lambda_i \text{在约束条件中的列} = \begin{bmatrix} 7x_3 + 5x_4 \\ 0 \\ 1 \end{bmatrix}$$

这说明对应于极点 $\begin{bmatrix} x_3 \\ x_4 \end{bmatrix}$ 的 λ_i 的价格是

$$\boldsymbol{c}_{\mathrm{BV}}^{\mathrm{T}} \boldsymbol{B}_0^{-1} \begin{bmatrix} 7x_3 + 5x_4 \\ 0 \\ 1 \end{bmatrix} - (70x_3 + 60x_4) = -70x_3 - 60x_4$$

由于 $\begin{bmatrix} x_3 \\ x_4 \end{bmatrix}$ 需要满足约束条件集合 2(或工厂 2 的约束条件)，所以具有最大负价格的 λ_i 将是与下列 LP 最优解的极点相关的加权。

工厂 2 的子问题如下：

$$\begin{aligned} \min \quad & z = -70x_3 - 60x_4 \\ \text{s.t.} \quad & 3x_3 + 2x_4 \leqslant 15 \\ & x_3 + x_4 \leqslant 4 \\ & x_3, x_4 \geqslant 0 \end{aligned}$$

求解工厂 2 的子问题，得解 $z = -280, x_3 = 4, x_4 = 0$。这意味着，与极点 $\boldsymbol{Q}_2 = \begin{bmatrix} 4 \\ 0 \end{bmatrix}$ 相关的 λ_2 在所有 λ_i 中具有最大负价格。由于 μ_4 的价格比 λ_2 还要负，所以把 μ_4 换入基。

μ_4 在单纯形表 0 中的列是

$$\boldsymbol{B}_0^{-1} \begin{bmatrix} 8x_1 + 6x_2 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 60 \\ 1 \\ 0 \end{bmatrix}$$

单纯形表 0 的右端项是

$$\boldsymbol{B}_0^{-1}\boldsymbol{b} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 80 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 80 \\ 1 \\ 1 \end{bmatrix}$$

比值测试表明，μ_4 应在第二个约束条件中换入基。因此 BV(1)=$\{s_1,\mu_4,\lambda_1\}$。由于

$$\boldsymbol{E}_0 = \begin{bmatrix} 1 & -60 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \boldsymbol{B}_1^{-1} = \boldsymbol{E}_0\boldsymbol{B}_0^{-1} = \begin{bmatrix} 1 & -60 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

所以 μ_4 的目标函数系数是 800。根据

$$c_{BV}^{T}B_1^{-1}=\begin{bmatrix}0 & 800 & 0\end{bmatrix}\begin{bmatrix}1 & -60 & 0\\0 & 1 & 0\\0 & 0 & 1\end{bmatrix}=\begin{bmatrix}0 & 800 & 0\end{bmatrix}$$

可以求出影子价格的新集合。接下来求当前单纯形表中具有最大负价格的加权。和前面一样，先求当前单纯形表的工厂 1 和工厂 2 的子问题，不断求出新的影子价格，直至得不到一个 μ_i 和 λ_i 具有有利的价格（子问题目标函数值大于或等于 0）。

最终得到限制主问题的最优解是 $\lambda_3=1,\mu_4=1,\lambda_1=0$，其他加权值都为 0。

下面把约束条件集合 1 可行域的表达式作为其极点的凸组合，来确定 $\begin{bmatrix}x_1\\x_2\end{bmatrix}$ 的最优值

$$\begin{bmatrix}x_1\\x_2\end{bmatrix}=0\boldsymbol{P}_1+0\boldsymbol{P}_2+0\boldsymbol{P}_3+\boldsymbol{P}_4=\begin{bmatrix}0\\10\end{bmatrix}$$

同样地，把约束条件集合 2 可行域的表达式作为其极点的凸组合，来确定 $\begin{bmatrix}x_3\\x_4\end{bmatrix}$ 的最优值

$$\begin{bmatrix}x_3\\x_4\end{bmatrix}=0\boldsymbol{Q}_1+0\boldsymbol{Q}_2+\boldsymbol{Q}_3=\begin{bmatrix}0\\4\end{bmatrix}$$

所以该问题的最优解是 $x_2=10,x_4=4,x_1=x_3=0$，即在工厂 1 生产 10 吨钢材 2，在工厂 2 生产 4 吨钢材 2，该公司能达到最大利润值 1040。

这里用 Python 调用 Gurobi 来验证上面的解，代码如下：

PythonGurobiDWExample

```
DW_model = Model('DW Decomposition')
mu = {}
lam = {}
s = {}
s[1] = DW_model.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS ,name= 's
       _1')
for i in range(4):
    mu[i+1] = DW_model.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS ,
       name= 'mu_' + str(i+1))
for i in range(3):
    lam[i+1] = DW_model.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS ,
       name= 'lam_' + str(i+1))

obj = LinExpr(0)
```

```
obj = 360 * mu[2] + 660 * mu[3] + 800 * mu[4] + 280 * lam[2] + 240 * lam[3]
DW_model.setObjective(obj, GRB.MAXIMIZE)
DW_model.addConstr(32*mu[2] + 52*mu[3] + 60*mu[4] + 27*lam[2] + 20*lam[3] +
      s[1] == 80, name = 'c1')
DW_model.addConstr(mu[1] + mu[2] + mu[3] + mu[4] == 1, name = 'c2')
DW_model.addConstr(lam[1] + lam[2] + lam[3] == 1, name = 'c3')
DW_model.optimize()

for var in DW_model.getVars():
    if(var.x > 0):
        print(var.varName, '\t', var.x)
```

得到的解如下。

PythonGurobiDWExampleOutput

```
Solved in 3 iterations and 0.02 seconds
Optimal objective  1.040000000e+03
mu_4     1.0
lam_3    1.0
```

可以看到，目标函数是相同的，也是 1040。并且最优解是 $\mu_4 = 1, \lambda_3 = 1$。跟上面的解也是相同的。

16.4 Dantzig-Wolfe 分解求解大规模混合整数规划

DW 分解算法的主要优点是将原问题分解成几个规模较小的子问题。而求解几个较小的 LP 问题通常比求解一个大型 LP 问题要容易得多。模型 (16.1) 是一个非常小的算例。如果说模型规模比较大的时候，比如对于 100 个客户点的 TSP，其变量个数要成千上万个，因此问题的可行域的极点数量就非常多。要想一开始就全部穷举出所有的极点就不太现实。我们观察到，模型 (16.12) 非常有规律，每一列都对应一个极点，比如 μ_1 就对应极点 $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ 而 μ_2 就对应极点 $\begin{bmatrix} 4 \\ 0 \end{bmatrix}$。每一个极点，例如 $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ 和 $\begin{bmatrix} 4 \\ 0 \end{bmatrix}$ 其实就是子问题的一个基可行解。所以只要找到每个子问题的可行域的所有极点（也就是该子问题的所有基可行解），DW 分解之后的完整的等价形式就可以写出来。规模较大时，不用完整地列举出所有的极点，也能找到问题的等价形式。该方法就是使用列生成算法。我们之前介绍过，列生成算法其实就是要找到将要进基的变量（一个进基变量就对应一个极点），因此我们用列生成的办法，将那些检验数为正（max 问题）的极点（也就是列）添加进限制性主问题 RMP，这就可以实现求解大规模线性规划或者整数规划的目的。

我们就以模型 (16.1) 为例，一步一步详细地展示这个过程。

16.4.1 两阶段法实现 Dantzig-Wolfe 分解算法介绍

首先将问题初始化，一般情况下我们需要用启发式算法生成一些初始的列。但是如果我们没有初始的列怎么办呢？文献（kalvelagen，2003）提供了一种用两阶段法求解的思路。两阶段法是线性规划求解中为人熟知的方法。

由于有凸组合约束和中心约束，因此我们刚开始添加的列可能会违反中心约束。因此我们可以在第 1 阶段引入人工变量，目标函数是最小化人工变量的和。引入人工变量的目的是判断该问题是否有可行解，如果第 1 阶段最小化人工变量的和的目标函数到达了 0，就说明该问题有解。我们就可以删去人工变量，进行第 2 阶段的算法，最终得到原问题的最优解。

本问题中，中心约束为

$$\sum_{i\in I}\alpha_i\mu_i+\sum_{j\in J}\beta_j\lambda_j\leqslant b \tag{16.13}$$

的形式，我们可以引入一个单独的人工变量 $x_a\geqslant 0$，并且将约束改写为

$$\sum_{i\in I}\alpha_i\mu_i+\sum_{j\in J}\beta_j\lambda_j-x_a\leqslant b \tag{16.14}$$

设置第 1 阶段的目标函数为

$$\min \quad x_a \tag{16.15}$$

此时，子问题产生的变量在目标函数中的系数全部为 0。当第 1 阶段的目标函数 (16.15) 为 0 时，我们开启第 2 阶段的算法。将人工变量的值固定为 0，继续迭代算法即可。

16.4.2 第 1 阶段

初始化问题的时候模型是空列。我们添加人工变量 s，将主问题初始化为

$$\max \quad z= \quad s \tag{16.16}$$

$$\text{s.t.} \quad -s \leqslant 80 \quad \to \pi_1 \tag{16.17}$$

$$\text{Null} = \text{Null} \quad \to \pi_2 \tag{16.18}$$

$$\text{Null} = \text{Null} \quad \to \pi_3 \tag{16.19}$$

$$s \geqslant 0 \tag{16.20}$$

$$\tag{16.21}$$

1. Iteration 0

模型 (16.16) 中，人工变量 s 就是为了算法能够进行下去才引入的，2 个凸组合的约束也需要初始化。我们先将其初始化为 Null。

相应的初始化代码如下。

DWInstancePhase1Step0

```
RMP = Model('DW Master Problem')

#  RMP.setParam("OutputFlag", 0)

#------------------------ start initialize RMP-----------------------

#  initialize column : Null column

row_num = 3

#  because vars in RMP is added into RMP dynamicly, thus we creat an array
      # to store the variable set
rmp_vars = []
rmp_vars.append(RMP.addVar(lb=0.0
                           , ub=1000
                           , obj= 1
                           , vtype=GRB.CONTINUOUS   #  GRB.BINARY, GRB.INTEGER
                           , name='s_1'
                           , column=None
                          ))

rmp_cons = []
var_names = []
col = [-1, 0.001, 0.001]
var_names.append('s_0')

#  column to add constraints into RMP
rmp_cons.append(RMP.addConstr(lhs = rmp_vars[0] * col[0]
                                  , sense= "<="
                                  , rhs= 80
                                  , name='rmp_con_central'
                                 )
               )

rmp_cons.append(RMP.addConstr(lhs = rmp_vars[0] * col[1]
                                  , sense= "=="
                                  , rhs= 1
                                  , name='rmp_con_convex_combine_mu'
                                 )
               )

```

```
rmp_cons.append(RMP.addConstr(lhs = rmp_vars[0] * col[2]
                                , sense= "=="
                                , rhs= 1
                                , name='rmp_con_convex_combine_lam'
                               )
                )


RMP.setAttr("ModelSense", GRB.MAXIMIZE)
#------------------------  RMP initialize end----------------------
RMP.chgCoeff(rmp_cons[1], rmp_vars[0], 0.0)
RMP.chgCoeff(rmp_cons[2], rmp_vars[0], 0.0)
rmp_cons[1].setAttr('RHS', 0)
rmp_cons[2].setAttr('RHS', 0)

RMP.write('DW.lp')
RMP.optimize()
rmp_dual = RMP.getAttr("Pi", RMP.getConstrs())
print('rmp_dual = ', rmp_dual)
```

运行上述模型，得到对偶变量如下。

DWInstancePhase1Step0Output

```
Optimal objective  1.000000000e+03
rmp_dual =  [0.0, 0.0, 0.0]
```

对偶变量是 $\pi^{\mathrm{T}}=[0,0,0]$。这里有一个细节。我们得到的对偶变量全为 0，这样就会使得子问题的目标函数为 0。为了刚开始能够产生新列，我们需要将两个 Null 的凸组合约束 (16.18) 和 (16.19) 的对偶变量 π_2,π_3 初始化为一个非零的数。这里我们初始化为 $\pi_2=\pi_3=-1$。因此对偶变量为 $\pi^{\mathrm{T}}=[0,-1,-1]$。

接下来我们需要构建子问题。子问题就是获得检验数为正的列。但是在该阶段，我们的目标函数只能是 max s，因此所有的即将新加入的列在目标函数的系数全为 0。

对于子问题 1 产生的列，检验数（Reduced cost）的表达式就可以写为

$$
\begin{aligned}
c_j-\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{N}_j &= 0-\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\cdot\begin{bmatrix}8x_1^{\mathrm{i}}+6x_2^{\mathrm{i}}\\1\\0\end{bmatrix}\\
&= 0-\boldsymbol{\pi}^{\mathrm{T}}\cdot\begin{bmatrix}8x_1^{\mathrm{i}}+6x_2^{\mathrm{i}}\\1\\0\end{bmatrix}
\end{aligned}
$$

$$
\begin{aligned}
&= 0 - \begin{bmatrix} 0 & -1 & -1 \end{bmatrix} \begin{bmatrix} 8x_1^i + 6x_2^i \\ 1 \\ 0 \end{bmatrix} \\
&= 1
\end{aligned}
$$

因此，子问题 1 的模型可以写成

$$
\begin{aligned}
\max \quad & 1 \\
\text{s.t.} \quad & 3x_1^i + x_2^i \leqslant 12 \\
& 2x_1^i + x_2^i \leqslant 10 \\
& x_1^i, x_2^i \geqslant 0
\end{aligned}
$$

求解上述模型的代码如下。

DWInstancePhase1Step0SP1

```
SP1 = Model('SubProblem 1')
x = {}
for i in range(2):
    x[i+1] = SP1.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name= '
       x' + str(i+1))

SP1.setObjective(0 * x[1] + 0 * x[2] + 1, GRB.MAXIMIZE)
#  SP1.setObjective(90 * x[1] + 80 * x[2], GRB.MAXIMIZE)
SP1.addConstr(3 * x[1] + x[2]  <= 12)
SP1.addConstr(2 * x[1] + x[2]  <= 10)
SP1.optimize()
print('Objective = \t', SP1.ObjVal)
for var in SP1.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =       1.0
x1 =      0.0
x2 =      0.0
```

目标函数 $1 > 0$，因此有新列加入。由于此时是第 1 阶段，因此目标函数的系数为 0。然后我们构造列

$$
\begin{bmatrix} 0 \\ 8x_1^i + 6x_2^i \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 8 \times 0 + 6 \times 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \text{obj} \\ \text{central constraint} \\ \text{convex constraint 1} \\ \text{convex constraint 2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}
$$

此时主问题变化为

$$\max z = \quad s \tag{16.22}$$

$$-s \leqslant 80 \tag{16.23}$$

$$\mu_1 = 1 \tag{16.24}$$

$$\text{Null} = \text{Null} \tag{16.25}$$

$$\mu_1, \quad s \geqslant 0 \tag{16.26}$$

然后我们求解子问题 2。类似地，子问题 2 的定价问题可以写成（假设可行域极点的集合为 J）。

$$\begin{aligned} \max \quad & 1 \\ \text{s.t.} \quad & 3x_3^j + 2x_4^j \leqslant 15 \\ & x_3^j + x_4^j \leqslant 4 \\ & x_3^j, x_4^j \geqslant 0 \end{aligned}$$

求解该问题的代码如下。

DWInstancePhase1Step0SP2

```
SP2 = Model('SubProblem 2')
xx = {}
for i in range(2):
    xx[i+1] = SP2.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name=
        'xx' + str(i+1))

SP2.setObjective(0 * xx[1] + 0 * xx[2] + 1, GRB.MAXIMIZE)
SP2.addConstr(3 * xx[1] + 2 * xx[2]  <= 15)
SP2.addConstr(xx[1] + xx[2]  <= 4)
SP2.optimize()
print('Objective = \t', SP2.ObjVal)
for var in SP2.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =       1.0
xx1 =     0.0
xx2 =     0.0
```

目标函数 $1 > 0$，因此有新列加入。同样地，由于此时是第一阶段，因此目标函数的系

数为 0。然后我们构造列

$$\begin{bmatrix} 0 \\ 7x_3^j + 5x_4^j \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 7\times 0 + 5 \times 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \text{obj} \\ \text{central constraint} \\ \text{convex constraint 1} \\ \text{convex constraint 2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

此时主问题变化为

$$\max \quad z = \quad s \tag{16.27}$$

$$-s \leqslant 80 \tag{16.28}$$

$$\mu_1 \quad = 1 \tag{16.29}$$

$$\lambda_1 \quad = 1 \tag{16.30}$$

$$\mu_1,\ \lambda_1,\ s \geqslant 0 \tag{16.31}$$

2. Iteration 1

我们求解更新后的主问题代码如下。

DWInstancePhase1Step1RMP

```
new_columns = [[0, 0, 1, 0]
               ,[0, 0, 0, 1]
               ]
for i in range(2):
    rmp_col = Column(new_columns[i][1:], rmp_cons)
    rmp_vars.append(RMP.addVar(lb=0.0
                               , ub=GRB.INFINITY
                               , obj=new_columns[i][0]  # obj=new_obj_coef
                               , vtype=GRB.CONTINUOUS
                               , name='y'+ str(i+1) # name='y\_' + str(len(rmp\_
        vars))
                               , column=rmp_col
                              ))
#  RMP.remove(rmp_vars[0])
#  del rmp_vars[0]
RMP.update()
RMP.write('DW.lp')
RMP.optimize()
rmp_dual = RMP.getAttr("Pi", RMP.getConstrs())
print('rmp_dual = ', rmp_dual)
print('Objective = \t', RMP.ObjVal)
for var in RMP.getVars():
```

```
    print(var.varName, '= \t', var.x)

[Out]:
rmp_dual =  [0.0, 0.0, 0.0]
Objective =       1000.0
s_1 =     1000.0
y1 =      0.0
y2 =      0.0
```

新的对偶变量为 $\pi^{\mathrm{T}}=[0,0,0]$，目标函数为 1000，因此还需要继续进行第 1 阶段的迭代。此时两个凸组合约束已经非空，此时我们就使用真实的对偶变量的值即可。

此时根据对偶变量更新子问题 1 的目标函数。子问题 1 的目标函数更新为

$$\begin{aligned} c_j-\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{N}_j &= 0-\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\cdot\begin{bmatrix}8x_1^i+6x_2^i\\1\\0\end{bmatrix}\\ &= 0-\boldsymbol{\pi}^{\mathrm{T}}\cdot\begin{bmatrix}8x_1^i+6x_2^i\\1\\0\end{bmatrix}\\ &= 0-\begin{bmatrix}0&0&0\end{bmatrix}\begin{bmatrix}8x_1^i+6x_2^i\\1\\0\end{bmatrix}\\ &= 0 \end{aligned}$$

因此子问题变化为

$$\begin{aligned} \max\quad & 0\\ \text{s.t.}\quad & 3x_1^i+x_2^i\leqslant 12\\ & 2x_1^i+x_2^i\leqslant 10\\ & x_1^i,x_2^i\geqslant 0 \end{aligned}$$

目标函数为 0，因此没有新列加入。

对于子问题 2，目标函数也为 0。两个子问题都没有新的列产生。因此，第 1 阶段结束。然后将主问题中的人工变量删除，或者将其设置为 0。

16.4.3 第 2 阶段

首先我们设置第 1 阶段结束时的主问题中的人工变量为 0，主问题变化为

$$\max\quad z= \qquad s \tag{16.32}$$

$$-s \leqslant 80 \tag{16.33}$$

$$\mu_1 = 1 \tag{16.34}$$

$$\lambda_1 = 1 \tag{16.35}$$

$$\mu_1,\ \lambda_1,\ s \geqslant 0 \tag{16.36}$$

$$s = 0$$

首先求解该问题，代码如下。

DWInstancePhase2Step1RMP

```
new_columns = [[0, 0, 1, 0]
               ,[0, 0, 0, 1]
               ]
for i in range(2):
    rmp_col = Column(new_columns[i][1:], rmp_cons)
    rmp_vars.append(RMP.addVar(lb=0.0
                               , ub=GRB.INFINITY
                               , obj=new_columns[i][0]  #  obj=new_obj_coef
                               , vtype=GRB.CONTINUOUS
                               , name='y'+ str(i+1) #name='y_' + str(len(rmp_vars))
                               , column=rmp_col
                              ))
# RMP.remove(rmp_vars[0])
# del rmp_vars[0]
rmp_cons[1].setAttr('RHS', 1)
rmp_cons[2].setAttr('RHS', 1)
RMP.update()
RMP.write('DW.lp')
RMP.optimize()
rmp_dual = RMP.getAttr("Pi", RMP.getConstrs())
print('rmp_dual = ', rmp_dual)
print('Objective = \t', RMP.ObjVal)
for var in RMP.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
rmp_dual =  [0.0, 0.0, 0.0]
Objective =       -0.0
s_1 =     0.0
y1 =      1.0
y2 =      1.0
```

新的对偶变量为 $\boldsymbol{\pi}^{\mathrm{T}}=[0,0,0]$。

第 2 阶段的子问题形式有所变化。假设工厂 1 对应的子问题 1 的约束的可行域的极点（2 维）有 I 个，其中每一个极点 $i\in I$ 的坐标为 (x_1^i,x_2^i)，我们知道，子问题 1 的可行域内的任何一点都是由极点集合 I 的坐标线性组合而成的，那么最优解 (x_1^*,x_2^*) 同样也是由极点集合 I 中的所有点的线性组合表示的。继续上面的过程，用 μ_i 表示子问题 1 中的第 i 个极点在最优解中贡献的比例。则子问题 1 对应的比例 μ_i 对 (16.32) 的目标函数系数的贡献为

$$90x_1^i+80x_2^i=\text{对应决策变量在目标函数中的系数}$$

且对中心约束的约束系数贡献为

$$8x_1^i+6x_2^i=\text{对应决策变量在中央约束中的约束系数}$$

因此，检验数（Reduced cost）的表达式就可以写为

$$\begin{aligned}c_j-\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\boldsymbol{N}_j&=\left(90x_1^i+80x_2^i\right)-\boldsymbol{c}_B^{\mathrm{T}}\boldsymbol{B}^{-1}\cdot\begin{bmatrix}8x_1^i+6x_2^i\\1\\0\end{bmatrix}\\&=\left(90x_1^i+80x_2^i\right)-\boldsymbol{\pi}^{\mathrm{T}}\cdot\begin{bmatrix}8x_1^i+6x_2^i\\1\\0\end{bmatrix}\end{aligned}$$

因此，子问题 1 的模型可以写成

$$\begin{aligned}\max\quad&\left(90x_1^i+80x_2^i\right)-\boldsymbol{\pi}^{\mathrm{T}}\cdot\left[8x_1^i+6x_2^i,\quad 1,\quad 0\right]^{\mathrm{T}}\\\text{s.t.}\quad&3x_1^i+x_2^i\leqslant 12\\&2x_1^i+x_2^i\leqslant 10\\&x_1^i,x_2^i\geqslant 0\end{aligned}$$

其中，$\boldsymbol{\pi}$ 是主问题约束的对偶变量，$\boldsymbol{\pi}^{\mathrm{T}}=[\pi_1,\pi_2,\pi_3]$。同理，子问题 2 的定价问题可以写成（假设可行域极点的集合为 J）：

$$\begin{aligned}\max\quad&\left(70x_3^j+60x_4^j\right)-\boldsymbol{\pi}^{\mathrm{T}}\cdot\left[7x_3^j+5x_4^j,\quad 0,\quad 1\right]^{\mathrm{T}}\\\text{s.t.}\quad&3x_3^j+2x_4^j\leqslant 15\\&x_3^j+x_4^j\leqslant 4\\&x_3^j,x_4^j\geqslant 0\end{aligned}$$

1. Iteration 1

新的对偶变量为 $\pi^{\mathrm{T}}=[0,0,0]$。我们根据对偶变量更新子问题 1。子问题 1 更新为

$$\begin{aligned}\max \quad & (90x_1^i+80x_2^i)-\begin{bmatrix}0, & 0, & 0\end{bmatrix}\cdot\begin{bmatrix}8x_1^i+6x_2^i, & 1, & 0\end{bmatrix}^{\mathrm{T}}\\ & =90x_1^i+80x_2^i\\ \text{s.t.} \quad & 3x_1^i+x_2^i\leqslant 12\\ & 2x_1^i+x_2^i\leqslant 10\\ & x_1^i,x_2^i\geqslant 0\end{aligned}$$

用 Gurobi 求解该子问题，代码及结果如下。

DWInstancePhase2Step1SP1

```
SP1 = Model('SubProblem 1')
x = {}
for i in range(2):
    x[i+1] = SP1.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name= '
       x' + str(i+1))

SP1.setObjective(90 * x[1] + 80 * x[2], GRB.MAXIMIZE)
SP1.addConstr(3 * x[1] + x[2]  <= 12)
SP1.addConstr(2 * x[1] + x[2]  <= 10)
SP1.optimize()
print('Objective = \t', SP1.ObjVal)
for var in SP1.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =       800.0
x1 =      0.0
x2 =      10.0
```

子问题目标函数 $800>0$，因此可以产生新列。新列的目标函数系数为 $90x_1^i+80x_2^i=80\times 10=800$。然后我们构造列

$$\begin{bmatrix}90x_1^i+80x_2^i\\ 8x_1^i+6x_2^i\\ 1\\ 0\end{bmatrix}=\begin{bmatrix}800\\ 60\\ 1\\ 0\end{bmatrix}=\begin{bmatrix}\text{obj}\\ \text{central constraint}\\ \text{convex constraint 1}\\ \text{convex constraint 2}\end{bmatrix}$$

所以，主问题中就可以将该列加入，如下：

$$\max\ z= \quad 800\mu_2 \quad +s \tag{16.37}$$

$$
\begin{aligned}
60\mu_2 \quad & -s \leqslant 80 && (16.38)\\
\mu_1 + \quad \mu_2 \quad & = 1 && (16.39)\\
\lambda_1 \quad & = 1 && (16.40)\\
\mu_1, \quad \lambda_1, \ s & \geqslant 0 && (16.41)\\
s & = 0
\end{aligned}
$$

然后接着求解如下子问题 2：

$$
\begin{aligned}
\max \quad & \left(70x_3^j + 60x_4^j\right) - \left[0, \ 0, \ 0\right] \cdot \left[7x_3^j + 5x_4^j, \ 0, \ 1\right]^{\mathrm{T}} \\
\text{s.t.} \quad & 3x_3^j + 2x_4^j \leqslant 15 \\
& x_3^j + x_4^j \leqslant 4 \\
& x_3^j, x_4^j \geqslant 0
\end{aligned}
$$

用 Gurobi 求解该子问题，代码及结果如下：

DWInstancePhase2Step1SP2

```
SP2 = Model('SubProblem 2')
xx = {}
for i in range(2):
    xx[i+1] = SP2.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name=
       'xx' + str(i+1))

SP2.setObjective(70 * xx[1] + 60 * xx[2], GRB.MAXIMIZE)
SP2.addConstr(3 * xx[1] + 2 * xx[2]  <= 15)
SP2.addConstr(xx[1] + xx[2]  <= 4)
SP2.optimize()
print('Objective = \t', SP2.ObjVal)
for var in SP2.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =      280.0
xx1 =    4.0
xx2 =    0.0
```

子问题目标函数 $280 > 0$，因此可以产生新列。新列的目标函数系数为 $70x_3^j + 60x_4^j =$

$70 \times 4 = 280$。然后我们构造列

$$\begin{bmatrix} 70x_3^j + 60x_4^j \\ 7x_3^j + 5x_4^j \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 280 \\ 28 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \text{obj} \\ \text{central constraint} \\ \text{convex constraint 1} \\ \text{convex constraint 2} \end{bmatrix}$$

所以，主问题中就可以将该列加入。主问题模型变化为

$$\max \quad z = \quad 800\mu_2 \quad + \quad 280\lambda_2 + s \tag{16.42}$$

$$60\mu_2 \quad + \quad 28\lambda_2 - s \leqslant 80 \tag{16.43}$$

$$\mu_1 + \quad \mu_2 \qquad\qquad = 1 \tag{16.44}$$

$$\lambda_1 + \quad \lambda_2 \quad = 1 \tag{16.45}$$

$$\mu_1, \quad \mu_2, \ \lambda_1, \quad \lambda_2, \ s \geqslant 0 \tag{16.46}$$

$$s = 0$$

求解更新后的主问题的代码如下：

PythonGurobiDWExampleStep2RMP

```
new_columns = [[800, 60, 1, 0]
               ,[280, 28, 0, 1]
              ]
for i in range(2):
    rmp_col = Column(new_columns[i][1:], rmp_cons)
    rmp_vars.append(RMP.addVar(lb=0.0
                               , ub=1
                               , obj=new_columns[i][0]  # obj=new_obj_coef
                               , vtype=GRB.CONTINUOUS
                               , name='y' #name='y_' + str(len(rmp_vars))
                               , column=rmp_col
                              ))
RMP.remove(rmp_vars[0])   # remove slack variable
del rmp_vars[0]
RMP.update()              # update model
RMP.write('DW.lp')
RMP.optimize()
rmp_dual = RMP.getAttr("Pi", RMP.getConstrs())
print('rmp_dual = ', rmp_dual)

[Out]:
rmp_dual =  [10.0, 200.0, 0.0]
```

```
Objective =       1000.0
s_1 =     0.0
y1 =      0.0
y2 =      0.2857142857142858
y5 =      1.0
y6 =      0.7142857142857143
```

新的对偶变量为 $\pi^{\mathrm{T}}=[10,200,0]$。

2. Iteration 2

继续更新子问题 1，首先，目标函数更新为

$$\begin{aligned}\left(90x_1^i+80x_2^i\right)-\left[10,\ \ 200,\ \ 0\right]\cdot\left[8x_1^i+6x_2^i,\ \ 1,\ \ 0\right]^{\mathrm{T}}&=90x_1^i+80x_2^i-80x_1^i-60x_2^i-200\\&=10x_1^i+20x_2^i-200\end{aligned}$$

子问题 1 更新为

$$\begin{aligned}\max\quad & 10x_1^i+20x_2^i-200\\ \text{s.t.}\quad & 3x_1^i+x_2^i\leqslant 12\\ & 2x_1^i+x_2^i\leqslant 10\\ & x_1^i,x_2^i\geqslant 0\end{aligned}$$

求解该子问题的代码及结果如下：

DWInstancePhase2Step2SP1

```
SP1 = Model('SubProblem 1')
x = {}
for i in range(2):
    x[i+1] = SP1.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name= '
       x' + str(i+1))

SP1.setObjective(10 * x[1] + 20 * x[2] - 200, GRB.MAXIMIZE)
SP1.addConstr(3 * x[1] + x[2]  <= 12)
SP1.addConstr(2 * x[1] + x[2]  <= 10)
SP1.optimize()
print('Objective = \t', SP1.ObjVal)
for var in SP1.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =       0
x1 =      0.0
```

```
x2 =      10.0
```

因为目标函数为 0，也就是检验数为 0，所以不会产生新列。之后的迭代中，可以直接跳过子问题 1。

接着求解子问题 2。子问题 2 的目标函数更新为

$$(70x_3^j + 60x_4^j) - \begin{bmatrix}10, & 200, & 0\end{bmatrix} \cdot \begin{bmatrix}7x_3^j + 5x_4^j, & 0, & 1\end{bmatrix}^{\mathrm{T}} = 10x_4^j$$

子问题 2 更新为

$$\begin{aligned} \max \quad & 10x_4^j \\ \text{s.t.} \quad & 3x_3^j + 2x_4^j \leqslant 15 \\ & x_3^j + x_4^j \leqslant 4 \\ & x_3^j, x_4^j \geqslant 0 \end{aligned}$$

求解该子问题的代码及结果如下：

PythonGurobiDWExampleStep1SP2

```
SP2 = Model('SubProblem 2')
xx = {}
for i in range(2):
    xx[i+1] = SP2.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name=
       'xx' + str(i+1))

SP2.setObjective(0 * xx[1] + 10 * xx[2], GRB.MAXIMIZE)
SP2.addConstr(3 * xx[1] + 2 * xx[2]  <= 15)
SP2.addConstr(xx[1] + xx[2]  <= 4)
SP2.optimize()
print('Objective = \t', SP2.ObjVal)
for var in SP2.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =       40.0
xx1 =     0.0
xx2 =     4.0
```

子问题目标函数 $40 > 0$，因此可以产生新列。新列的目标函数系数为 $70x_3^j + 60x_4^j = 60 \times 4 = 240$。然后构造列

$$\begin{bmatrix}70x_3^j + 60x_4^j \\ 7x_3^j + 5x_4^j \\ 0 \\ 1\end{bmatrix} = \begin{bmatrix}240 \\ 20 \\ 0 \\ 1\end{bmatrix} = \begin{bmatrix}\text{obj} \\ \text{central constraint} \\ \text{convex constraint 1} \\ \text{convex constraint 2}\end{bmatrix}$$

所以，主问题中就可以将该列加入。主问题模型变化为

$$\max \quad z = \quad 800\mu_2 \quad +280\lambda_2+240\lambda_3+s \tag{16.47}$$

$$\text{s.t.} \quad 60\mu_2 \quad + 28\lambda_2+ 20\lambda_3-s \leqslant 80 \tag{16.48}$$

$$\mu_1+ \quad \mu_2 \quad = 1 \tag{16.49}$$

$$\lambda_1+ \quad \lambda_2+ \quad \lambda_3 \quad = 1 \tag{16.50}$$

$$\mu_1, \quad \mu_2, \ \lambda_1, \quad \lambda_2, \quad \lambda_3, \ s \geqslant 0 \tag{16.51}$$

$$s = 0$$

下面求解更新后的主问题，代码及结果如下：

DWInstancePhase2Step2RMP

```
new_columns = [[240, 20, 0, 1]
               ]
for i in range(1):
    rmp_col = Column(new_columns[i][1:], rmp_cons)
    rmp_vars.append(RMP.addVar(lb=0.0
                               , ub=GRB.INFINITY
                               , obj=new_columns[i][0]  # obj=new_obj_coef
                               , vtype=GRB.CONTINUOUS
                               , name='y'+ str(i+1) #name='y_' + str(len(rmp_vars))
                               , column=rmp_col
                              ))
# RMP.remove(rmp_vars[0])
# del rmp_vars[0]
RMP.update()
RMP.write('DW.lp')
RMP.optimize()
rmp_dual = RMP.getAttr("Pi", RMP.getConstrs())
print('rmp_dual = ', rmp_dual)
print('Objective = \t', RMP.ObjVal)
for var in RMP.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
rmp_dual =  [10.0, 0.0, 40.0]
Objective =       840.0
s_1 =     0.0
y1 =      0.25
y2 =      0.0
y1 =      0.75
```

```
y2 =      0.0
y1 =      1.0
```

新的对偶变量为 $\pi^{\mathrm{T}}=[10,0,40]$。

3. Iteration 3

由于之前子问题 1 已经没有新列产生，我们直接来看子问题 2。

子问题 2 的目标函数更新为

$$\left(70x_3^j+60x_4^j\right)-\left[10,\quad 0,\quad 40\right]\cdot\left[7x_3^j+5x_4^j,\quad 0,\quad 1\right]^{\mathrm{T}}=10x_4^j-40$$

子问题 2 更新为

$$\begin{aligned}\max\quad & 10x_4^j-40\\ \text{s.t.}\quad & 3x_3^j+2x_4^j\leqslant 15\\ & x_3^j+x_4^j\leqslant 4\\ & x_3^j,x_4^j\geqslant 0\end{aligned}$$

求解该子问题的代码及结果如下：

PythonGurobiDWExampleStep3SP2

```
SP2 = Model('SubProblem 2')
xx = {}
for i in range(2):
    xx[i+1] = SP2.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name=
       'xx' + str(i+1))

SP2.setObjective(0 * xx[1] + 10 * xx[2] - 40, GRB.MAXIMIZE)
SP2.addConstr(3 * xx[1] + 2 * xx[2]  <= 15)
SP2.addConstr(xx[1] + xx[2]  <= 4)
SP2.optimize()
print('Objective = \t', SP2.ObjVal)
for var in SP2.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =       -0.0
xx1 =     0.0
xx2 =     4.0
```

没有新列产生。迭代终止。最终的主问题变成

$$\max\quad z=\quad 800\mu_2\quad +280\lambda_2+240\lambda_3+s \tag{16.52}$$

$$60\mu_2 + 28\lambda_2 + 20\lambda_3 - s \leqslant 80 \tag{16.53}$$

$$\mu_1 + \mu_2 = 1 \tag{16.54}$$

$$\lambda_1 + \lambda_2 + \lambda_3 = 1 \tag{16.55}$$

$$\mu_1, \mu_2, \lambda_1, \lambda_2, \lambda_3, s \geqslant 0 \tag{16.56}$$

$$s = 0$$

求解该主问题的代码及结果如下：

DWInstancePhase2Step2RMPFinal

```
new_columns = [[240, 20, 0, 1]
               ]
for i in range(1):
    rmp_col = Column(new_columns[i][1:], rmp_cons)
    rmp_vars.append(RMP.addVar(lb=0.0
                               , ub=GRB.INFINITY
                               , obj=new_columns[i][0]  # obj=new_obj_coef
                               , vtype=GRB.CONTINUOUS
                               , name='y'+ str(i+7) #name='y_' + str(len(rmp_vars))
                               , column=rmp_col
                              ))
# RMP.remove(rmp_vars[0])
# del rmp_vars[0]
RMP.update()
RMP.write('DW.lp')
RMP.optimize()
rmp_dual = RMP.getAttr("Pi", RMP.getConstrs())
print('rmp_dual = ', rmp_dual)
print('Objective = \t', RMP.ObjVal)
for var in RMP.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
rmp_dual =  [12.0, 80.0, 0.0]
Objective =       1040.0
s_1 =     0.0
y1 =      0.0
y2 =      0.0
y5 =      1.0
y6 =      0.0
y7 =      1.0
```

因此，得到最优值为 1040。其中 y_5 对应的极点为 [0, 10]，是属于子问题 1 产生的极点，因此 $[x_1, x_2] = y_5 \cdot [0, 10] = [0, 10]$。$y_7$ 对应的极点为 [0, 4]，是属于子问题 2 产生的极点，因此 $[x_3, x_4] = y_7 \cdot [0, 4] = [0, 4]$。因此原问题的最优解为 $[x_1, x_2, x_3, x_4] = [0, 10, 0, 4]$。

至此，用列生成的方法结合 Gurobi 的 Python 接口，实现了 Dantzig-Wolfe 分解算法的详细迭代过程。在实际大规模问题中，我们可以将上述过程写成循环，以实现自动化运行整个算法的目的。

在后面的章节中将利用上述思路，用 Dantzig-Wolfe 分解算法求解多商品网络流问题（MCNF）以及带时间窗的车辆路径规划问题（Vehicle Routing Problem with Time Windows，VRPTW）。

16.5 Python 调用 Gurobi 实现 Dantzig-Wolfe 分解求解多商品流运输问题

16.5.1 多商品网络流模型的区块结构

我们以多商品网络流（Multicommodity Network Flow，MCNF）问题为例，阐述用 DW 分解算法求解该问题的整体思路。

首先参照第 2 章相关内容，给出 MCNF 的模型（点-弧模型，Node-Arc Model）如下：

$$\min \quad \sum_{k\in K}\sum_{k} c_{ij}^k x_{ij}^k \tag{16.57}$$

$$\sum_{j\in V} x_{ij}^k - \sum_{i\in V} x_{ji}^k = \begin{cases} d_k, & i = s_k,\ \forall i \in V \\ -d_k, & i = t_k,\ \forall i \in V \\ 0, & \text{其他},\ \forall i \in V \end{cases} \tag{16.58}$$

$$\sum_{k\in K} x_{ij}^k \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{16.59}$$

$$x_{ij}^k \geqslant 0, \qquad \forall (i,j) \in A, k \in K \tag{16.60}$$

为了方便分解，我们将模型改写为

$$\min \quad \sum_{k\in K}\sum_{(i,j)\in A} c_{ij}^k x_{ij}^k \tag{16.61}$$

$$\sum_{j\in V} x_{ij}^k - \sum_{i\in V} x_{ji}^k = b_i^k, \qquad \forall i \in V,\ \forall k \in K \tag{16.62}$$

$$\sum_{k} x_{ij}^k \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{16.63}$$

$$x_{ij}^k \geqslant 0, \qquad \forall (i,j) \in A,\ \forall k \in K \tag{16.64}$$

其中，当 $i = s_k$ 时，$b_i^k = d_k$；当 $i = t_k$ 时，$b_i^k = -d_k$；其余情况 $b_i^k = 0$。

观察得知约束 (16.62) 中是针对 $\forall k \in K$ 的，而约束 (16.63) 却只针对 $\forall (i,j) \in A$，因此可以判断，该模型是符合块角矩阵形式的。其中，约束 (16.63) 是每一条弧 (i,j) 上所有商品流 k 的加和。而约束 (16.62) 则可以按照商品流序号 k 分成 $|K|$ 个块。正好可以使用 DW 分解算法来求解。

也许上述描述对于初学者来讲并不是很清楚，那么我们继续以文献（Cappanera and Scaparra，2011）中的例子来讲解。示例网络如图 16.4所示。

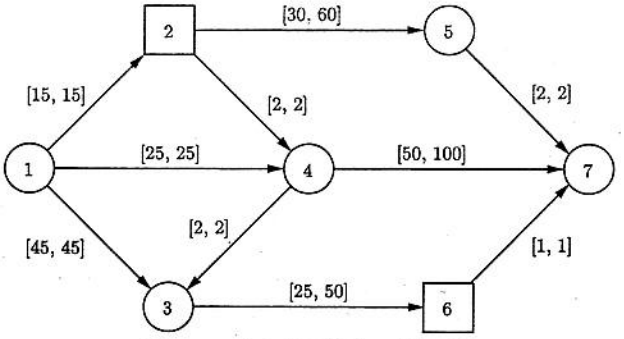


图 16.4 多商品网络流: 示例网络

假设我们考虑有 2 个商品流，分别表示如下：

（1）commodity 1：[1, 7, 25]；

（2）commodity 2：[2, 6, 2]；

其中，[1, 7, 25] 表示起点为 1、终点为 7、需求是 25 单位。我们继续用 Excel 将其具体模型写出来。整理可以得出如图 16.5所示的表格。

根据图 16.5中的信息，我们就很清楚地看到 MCNF 的区块结构。

16.5.2 多商品流运输问题: Dantzig-Wolfe 分解求解

本章的实践中我们不做 MCNF 的具体实现，我们仅做多商品流运输问题（Multicommodity Transportation Problem, MCTP）的实现。回顾第 2 章的内容，我们知道 MCNF 和 MCTP 的区别在于，MCTP 中货物并没有网络中间点的转运，而是直接从起点运输到终点。我们首先在此给出 MCTP 的数学模型如下：

$$\min \quad \sum_{k\in K}\sum_{(i,j)\in A} c_{ij}^k x_{ij}^k \tag{16.65}$$

$$\sum_{j\in C} x_{ij}^k = s_i^k \qquad \forall i \in S,\ \forall k \in K \tag{16.66}$$

$$\sum_{i\in S} x_{ij}^k = d_j^k, \qquad \forall j \in C,\ \forall k \in K \tag{16.67}$$

$$\sum_{k\in K} x_{ij}^k \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{16.68}$$

$$x_{ij}^k \geqslant 0, \qquad \forall (i,j) \in A,\ \forall k \in K \tag{16.69}$$

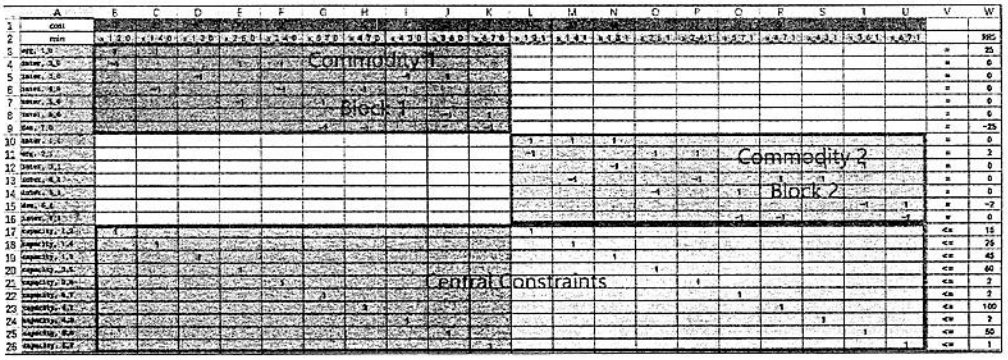


图 16.5 MCNF 模型的表格形式：2 个区块

可以看出，约束 (16.66) 和约束 (16.67) 都是带有 $\forall k \in K$ 的，是很明显的分块矩阵，而约束 (16.68) 则是中心约束。

我们将模型 (16.65) 按照商品流 $k \in K$ 分解为 $|K|$ 个子问题，每一个子问题中我们固定了商品流 k，因此子问题中下标 k 可以省略，子问题的具体形式为

$$\min \quad \sum_{(i,j)\in A} c_{ij} x_{ij} \tag{16.70}$$

$$\text{s.t.} \quad \sum_{j\in C} x_{ij} = s_i, \qquad \forall i \in S \tag{16.71}$$

$$\sum_{i\in S} x_{ij} = d_j, \qquad \forall j \in C \tag{16.72}$$

$$x_{ij} \geqslant 0, \qquad \forall (i,j) \in A \tag{16.73}$$

而主问题就只保留中心约束，即为

$$\min \quad \sum_{k\in K} \sum_{(i,j)\in A} c_{ij}^k x_{ij}^k \tag{16.74}$$

$$\text{s.t.} \quad \sum_{k\in K} x_{ij}^k \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{16.75}$$

$$x_{ij}^k \geqslant 0, \qquad \forall (i,j) \in A,\ \forall k \in K \tag{16.76}$$

但是由于我们使用了 DW 分解，就不能继续沿用原变量 x_{ij}^k 了，我们这里把 DW 分解产生的列对应的变量叫作 $\boldsymbol{\lambda}$，而第 k 个子问题产生的第 v 个极点对应的列产生的变量则为 $\lambda_{k,v}$。假设我们使用启发式算法或者两阶段法首先初始化了主问题，产生了一些初始的列 $\boldsymbol{y}_0$（或者人工变量），这些列对应的约束系数矩阵为 $\boldsymbol{B}_0$，则初始主问题可以写成

$$\min \quad \boldsymbol{c}_0^{\mathrm{T}} \boldsymbol{y}_0 \tag{16.77}$$

$$\boldsymbol{B}_{ij}^{0}\boldsymbol{y}_0 \leqslant u_{ij}, \qquad \forall (i,j) \in A \tag{16.78}$$

$$\text{Null} = \text{Null} \qquad \forall k \in K \rightarrow \text{凸组合约束} \tag{16.79}$$

$$\boldsymbol{y}_0 \geqslant \boldsymbol{0} \tag{16.80}$$

当产生了一系列 $\boldsymbol{\lambda}$ 以后，我们的主问题中的凸组合约束（convex combination constraints）就不再为空。我们用 $\bar{x}$ 表示子问题的解（或者子问题的极点），因此主问题就更新为

$$\min_{\boldsymbol{\lambda}} \quad \boldsymbol{c}_0^{\mathrm{T}}\boldsymbol{y}_0 + \sum_{k\in K}\sum_{v=1}^{\mathrm{SP}_k}\left(\boldsymbol{c}_k^{\mathrm{T}}\bar{\boldsymbol{x}}_k^{(v)}\right)\lambda_{k,v} \tag{16.81}$$

$$\text{s.t.} \quad \boldsymbol{B}_{ij}^{0}\boldsymbol{y}_0 + \sum_{k\in K}\sum_{v=1}^{\mathrm{SP}_k}\left(\boldsymbol{B}_k\bar{\boldsymbol{x}}_k^{(v)}\right)\lambda_{k,v} \leqslant u_{ij}, \quad \forall (i,j) \in A \tag{16.82}$$

$$\sum_{v=1}^{\mathrm{SP}_k}\delta_{k,v}\lambda_{k,v} = 1, \qquad \forall k \in K \tag{16.83}$$

$$\boldsymbol{y}_0 \geqslant \boldsymbol{0} \tag{16.84}$$

$$\lambda_{k,v} \geqslant 0, \qquad \forall k \in K, \forall v \in \mathrm{SP}_k \tag{16.85}$$

其中，SP_k 表示由第 k 个子问题产生的极点的个数或者极射线的集合的大小。$\boldsymbol{c}_k^{\mathrm{T}}\bar{\boldsymbol{x}}_k^{(v)}$ 表示第 k 个子问题产生的第 v 个极点或者极射线对应的解产生的总成本（一个极点就对应子问题的一个基可行解，其实就是子问题中 x_{ij} 的取值 $\bar{x}_{ij}$，这里为了简略直接将子问题 k 的解写成了 $\bar{\boldsymbol{x}}_k$，但是其实 $\bar{\boldsymbol{x}}_k$ 是一个 $|A|$ 维的向量）。也就是这个极点 j（或者解）如果被选择，会对主问题的目标函数贡献多少。而这个极点对应的主问题的决策变量（或者主问题的列）就是 $\lambda_{k,v}$；并且有

$$\boldsymbol{c}_k^{\mathrm{T}}\bar{\boldsymbol{x}}_k^{(v)} = \sum_{(i,j)\in A} c_{ij}\bar{x}_{ij}$$

这里 x_{ij} 是第 k 个子问题中的决策变量，$\bar{x}_{ij}$ 就是子问题的一个解（或者极点）。

$\lambda_{k,v}$ 在中心约束中的约束系数就是对应第 k 个子问题对弧段 (i,j) 贡献的总流量，也就是 $\boldsymbol{B}_k\bar{\boldsymbol{x}}_k^{(v)}$，且在主问题第 (i,j) 个中心约束中，$\lambda_{k,v}$ 的约束系数为

$$\lambda_{k,v}\text{在中央约束中的约束系数} = \boldsymbol{B}_k\bar{\boldsymbol{x}}_k^{(v)} = \bar{x}_{ij}$$

因为只有 x_{ij} 才是弧 (i,j) 上的流量。

另外，参数 $\delta_{k,v}$ 是 $\lambda_{k,v}$ 在凸组合约束中的系数。$\lambda_{k,v}$ 只有在对应的第 k 个子问题对应的凸组合约束中，系数才为 1，其他情况均为 0。也就是

$$\delta_{k,v} = \begin{cases} 1, & k = k_0 \\ 0, & \text{其他} \end{cases}$$

其中，k_0 是当前子问题的 ID。

因此，主问题的形式可以进一步写成更具体的形式，如下：

$$\min_{\lambda} \quad c_0^{\mathrm{T}} y_0 + \sum_{k\in K}\sum_{v=1}^{\mathrm{SP}_k}\left(\sum_{(i,j)\in A} c_{ij}^k \bar{x}_{ij}^{(v)}\right)\lambda_{k,v} \tag{16.86}$$

$$B_{ij}^0 y_0 + \sum_{k\in K}\sum_{v=1}^{\mathrm{SP}_k}\left(\bar{x}_{ij}^{(v)}\right)\lambda_{k,v} \leqslant u_{ij}, \qquad \forall (i,j)\in A \tag{16.87}$$

$$\sum_{v=1}^{\mathrm{SP}_k}\delta_{k,v}\lambda_{k,v} = 1, \qquad \forall k \in K \tag{16.88}$$

$$y_0 \geqslant \mathbf{0} \tag{16.89}$$

$$\lambda_{k,v} \geqslant 0 \tag{16.90}$$

如果我们将其展开，就是下面的形式：

$$\begin{array}{llllll}
\min\limits_{\lambda} & c_0^{\mathrm{T}}y_0+\left(\sum\limits_{(i,j)} c_{ij}^1\bar{x}_{ij}^{(1)}\right)\lambda_{1,1}+\left(\sum\limits_{(i,j)} c_{ij}^1\bar{x}_{ij}^{(2)}\right)\lambda_{1,2}+\cdots+\left(\sum\limits_{(i,j)} c_{ij}^2\bar{x}_{ij}^{(1)}\right)\lambda_{2,1}+\left(\sum\limits_{(i,j)} c_{ij}^2\bar{x}_{ij}^{(2)}\right)\lambda_{2,2}+\cdots & & & & \\
& y_0 \ +\left(\bar{x}_{12}^{(1)}\right)\lambda_{1,1} & +\left(\bar{x}_{12}^{(2)}\right)\lambda_{1,2} & +\cdots+\left(\bar{x}_{12}^{(1)}\right)\lambda_{2,1} & +\left(\bar{x}_{12}^{(2)}\right)\lambda_{2,2} & +\cdots\leqslant u_{12} \\
& y_0 \ +\left(\bar{x}_{23}^{(1)}\right)\lambda_{1,1} & +\left(\bar{x}_{23}^{(2)}\right)\lambda_{1,2} & +\cdots+\left(\bar{x}_{23}^{(1)}\right)\lambda_{2,1} & +\left(\bar{x}_{23}^{(2)}\right)\lambda_{2,2} & +\cdots\leqslant u_{23} \\
& \vdots \quad \vdots & \vdots & & \vdots & \vdots \quad \vdots \\
& \lambda_{1,1} & +\lambda_{1,2} & +\cdots & & =1 \\
& & & \lambda_{2,1} & +\lambda_{2,2} & +\cdots=1 \\
& y_0, \quad \lambda_{1,1}, & \lambda_{1,2}, & \cdots\ \lambda_{2,1}, & \lambda_{2,2}, & \cdots\geqslant 0 \\
& & & & & y_0 = \mathbf{0}
\end{array}$$

由于子问题规模也不小，因此极点数量较多，直接一开始就列举出来是不现实的。要用列生成的办法来做。

首先来推导子问题的目标函数。子问题就是找到检验数为负的列。用 π_1^{ij} 表示约束 (16.87) 的对偶变量，用 π_2^k 表示约束 (16.88) 的对偶变量，则主问题中一列的检验数为

$$\sigma_{k,v} = \left(c_k^{\mathrm{T}}\bar{x}_k^{(v)}\right) - \pi^{\mathrm{T}}\begin{bmatrix} B_k\bar{x}_k^{(v)} \\ \delta_{k,v}\end{bmatrix} \tag{16.91}$$

$$= \left(c_k^{\mathrm{T}} - \pi_1^{\mathrm{T}}B_k\right)\bar{x}_k^{(v)} - \pi_2^{\mathrm{T}}\delta_{k,v} \tag{16.92}$$

$$= \left(\sum_{(i,j)\in A}\left(c_{ij}^k - \pi_1^{ij}\right)\bar{x}_{ij}^{(v)}\right) - \pi_2^k \tag{16.93}$$

因此，子问题就更新为

$$\min_{x}\ \sigma_k = \left(\sum_{(i,j)\in A}\left(c_{ij} - \pi_1^{ij}\right)x_{ij}\right) - \pi_2 \tag{16.94}$$

$$\text{s.t.} \sum_{j\in C} x_{ij} = s_i, \qquad \forall i \in S \tag{16.95}$$

$$\sum_{i\in S} x_{ij} = d_j, \qquad \forall j \in C \tag{16.96}$$

$$x_{ij} \geqslant 0, \qquad \forall (i,j) \in A \tag{16.97}$$

若子问题目标函数为负，则可以产生如下新列（假设针对子问题 1，此时 $k=1$）：

$$\text{new column} = \begin{bmatrix} \sum_{(i,j)} c_{ij}^1 \bar{x}_{ij}^{(v)} \\ \bar{x}_{12}^{(v)} \\ \bar{x}_{23}^{(v)} \\ \vdots \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \cdot \lambda_{1,v} = \begin{bmatrix} \text{目标函数中的系数} \\ \text{弧}(1,2)\text{对应中央约束中的约束系数} \\ \text{弧}(2,3)\text{对应中央约束中的约束系数} \\ \vdots \\ \text{子问题 1 对应凸约束中的约束系数} \\ \text{子问题 2 对应凸约束中的约束系数} \\ \vdots \\ \text{子问题 } K \text{ 对应凸约束中的约束系数} \end{bmatrix} \cdot \lambda_{1,v}$$

16.5.3 Python 调用 Gurobi 实现 Dantzig-Wolfe 分解求解多商品流运输问题

本节尝试用 Python 调用 Gurobi 实现 Dantzig-Wolfe 分解求解 MCTP。具体来讲，就是结合 16.4 节介绍的两阶段法，结合列生成的思想来实现整个 Dantzig-Wolfe 分解算法的框架。最后用一个实际的算例来测试算法，以展示算法的正确性。本章代码很大程度上参考了杉数科技算法工程师伍健[①] 共享在 github 上的代码。该代码实现的算法也正是 16.5.2 节介绍的 MCTP 的模型，具体细节参考自文献（Kalvelagen，2003）。

16.5.4 完整代码

完整代码如下：

MCTP.py

```
from __future__ import division, print_function
from gurobipy import *

class MCTP:
    def __init__(self):
        # initialize data
        self.NumOrg = 0
        self.NumDes = 0
        self.NumProduct = 0
```

① 伍健 E-mail 地址为 wujianjack2@163.com。

```
        self.Supply = []
        self.Demand = []
        self.Capacity = []
        self.Cost = []

        # initialize variables and constraints in RMP
        self.CapacityCons = []    # capacity constraints
        self.var_lambda = []      # decision variable of RMP, i.e., columns

        # initialize variables in subproblem
        self.var_x = []

        # initialize parameters
        self.Iter = 0
        self.dual_convexCons = 1
        # 凸组合约束的对偶变量，更新子问题目标函数使用
        # 这里赋予一个初值1，为了在第一次迭代的时候，子问题目标函数能小于0，
        # 能有新的检验数为负的列产生
        self.dual_CapacityCons = []
        # 容量约束的对偶变量，更新子问题目标函数使用
        self.SP_totalCost = []
        # 子问题的解产生的总运输成本，在添加新列的时候，是新变量的目标函数
        # 系数

    def readData(self, filename):
        # input data
        with open(filename, "r") as data:
            self.NumOrg = int(data.readline())
            self.NumDes = int(data.readline())
            self.NumProduct = int(data.readline())

            for i in range(self.NumOrg):
                col = data.readline().split()
                SupplyData = []
                for k in range(self.NumProduct):
                    SupplyData.append(float(col[k]))
                self.Supply.append(SupplyData)

            for j in range(self.NumDes):
                col = data.readline().split()
                DemandData = []
                for k in range(self.NumProduct):
```

```
                    DemandData.append(float(col[k]))
                self.Demand.append(DemandData)

            for i in range(self.NumOrg):
                col = data.readline().split()
                CapacityData = []
                for j in range(self.NumDes):
                    CapacityData.append(float(col[j]))
                self.Capacity.append(CapacityData)

            for i in range(self.NumOrg):
                CostData = []
                for j in range(self.NumDes):
                    col = data.readline().split()
                    CostData_temp = []
                    for k in range(self.NumProduct):
                        CostData_temp.append(float(col[k]))
                    CostData.append(CostData_temp)
                self.Cost.append(CostData)

    def initializeModel(self):
        # initialize master problem and subproblem
        self.RMP = Model("RMP")
        self.subProblem = Model("subProblem")

        # close log information
        self.RMP.setParam("OutputFlag", 0)
        self.subProblem.setParam("OutputFlag", 0)

        # add initial artificial variable in RMP, in order to start the algorithm
        self.var_Artificial = self.RMP.addVar(lb = 0.0
                                               , ub = GRB.INFINITY
                                               , obj = 0.0
                                               , vtype = GRB.CONTINUOUS
                                               , name = 'Artificial'
                                              )

        # add temp capacity constraints in RMP, in order to start the algorithm
        for i in range(self.NumOrg):
            CapCons_temp = []
            for j in range(self.NumDes):
                CapCons_temp.append(self.RMP.addConstr(-self.var_Artificial
```

```
        <= self.Capacity[i][j], name = 'capacity cons'))
89          self.CapacityCons.append(CapCons_temp)
90
91      # initialize the convex combination constraints
92      self.convexCons = self.RMP.addConstr(1 * self.var_Artificial == 1,
        name = 'convex cons')
93
94      # add variables to subproblem, x is flow on arcs
95      for i in range(self.NumOrg):
96          var_x_array = []
97          for j in range(self.NumDes):
98              var_x_temp = []
99              for k in range(self.NumProduct):
100                 var_x_temp.append(self.subProblem.addVar(lb = 0.0
101                                                 , ub = GRB.INFINITY
102                                                 , obj = 0.0
103                                                 , vtype = GRB.CONTINUOUS))
104             var_x_array.append(var_x_temp)
105         self.var_x.append(var_x_array)
106
107     # add constraints supply to subproblem
108     for i in range(self.NumOrg):
109         for k in range(self.NumProduct):
110             self.subProblem.addConstr(quicksum(self.var_x[i][j][k] for j
        in range(self.NumDes)) \
111                                               == self.Supply[i][k], name
        = 'Supply_' + str(i) + '_' + str(k))
112
113     # add constraints demand for subproblem
114     for j in range(self.NumDes):
115         for k in range(self.NumProduct):
116             self.subProblem.addConstr(quicksum(self.var_x[i][j][k] for i
        in range(self.NumOrg)) \
117                                               == self.Demand[j][k], name
        = 'Demand_' + str(j) + '_' + str(k))
118     # export the initial RMP with artificial variable
119     self.RMP.write('initial_RMP.lp')
120
121 def optimizePhase_1(self):
122     # initialize parameters
123     for i in range(self.NumOrg):
124         dual_capacity_temp = [0.0] * self.NumDes
```

```
            self.dual_CapacityCons.append(dual_capacity_temp)

        obj_master_phase_1 = self.var_Artificial
        obj_sub_phase_1 = -quicksum(self.dual_CapacityCons[i][j] * self.var_
    x[i][j][k] \
                                    for i in range(self.NumOrg) \
                                    for j in range(self.NumDes) \
                                    for k in range(self.NumProduct)) - self.
    dual_convexCons

        # set objective for RMP of Phase 1
        self.RMP.setObjective(obj_master_phase_1, GRB.MINIMIZE)

        # set objective for subproblem of Phase 1
        self.subProblem.setObjective(obj_sub_phase_1, GRB.MINIMIZE)

        # in order to make initial model is feasible, we set initial convex
        # constraints to Null
        # and in later iteration, we set the RHS of convex constraint to 1
        self.RMP.chgCoeff(self.convexCons, self.var_Artificial, 0.0)

        # Phase 1 of Dantzig-Wolfe decomposition : to ensure the initial
        # model is feasible
        print(" --------- start Phase 1 optimization ---------  ")

        while True:
            print("Iter: ", self.Iter)
            # export the model and check whether it is correct
            self.RMP.write('solve_master.lp')
            # solve subproblem of phase 1
            self.subProblem.optimize()

            if self.subProblem.objval >= -1e-6:
                print("No new column will be generated, coz no negative
    reduced cost columns")
                break
            else:
                self.Iter =  self.Iter + 1

                # compute the total cost of subproblem solution
                # the total cost is the coefficient of RMP when new column is added
                totalCost_subProblem = sum(self.Cost[i][j][k] * self.var_x[i
```

```
][j][k].x \
                                    for i in range(self.NumOrg) \
                                    for j in range(self.NumDes) \
                                    for k in range(self.NumProduct))

            self.SP_totalCost.append(totalCost_subProblem)

            # update constraints in RMP
            col = Column()
            for i in range(self.NumOrg):
                for j in range(self.NumDes):
                    col.addTerms(sum(self.var_x[i][j][k].x for k in
range(self.NumProduct)), self.CapacityCons[i][j])

            col.addTerms(1.0, self.convexCons)

            # add decision variable lambda into RMP, i.e., extreme point
 # obtained from subproblems
            self.var_lambda.append(self.RMP.addVar(lb = 0.0
                                                   , ub = GRB.INFINITY
                                                   , obj = 0.0
                                                   , vtype = GRB.
CONTINUOUS
                                                   , name = "lam_phase1_
" + str(self.Iter)
                                                   , column = col))

            # solve RMP in Phase 1
            self.RMP.optimize()

            # update dual variables
            if self.RMP.objval <= 1e-6:
                print("---obj of phase 1 reaches 0, phase 1 ends---")
                break
            else:
                for i in range(self.NumOrg):
                    for j in range(self.NumDes):
                        self.dual_CapacityCons[i][j] = self.CapacityCons
[i][j].pi

                self.dual_convexCons = self.convexCons.pi

```

```
198                 # reset objective for subproblem in 'Phase 1'
199                 obj_sub_phase_1 = -quicksum(self.dual_CapacityCons[i][j] *
    self.var_x[i][j][k] \
200                                             for i in range(self.NumOrg) \
201                                             for j in range(self.NumDes) \
202                                             for k in range(self.NumProduct))
    - self.dual_convexCons
203
204                 self.subProblem.setObjective(obj_sub_phase_1, GRB.MINIMIZE)
205
206     def updateModelPhase_2(self):
207         # update model of Phase 2
208         print(" ----start update model of Phase 2 ----")
209
210         # set objective of RMP in Phase 2
211         obj_master_phase_2=quicksum(self.SP_totalCost[i]*self.var_lambda[i]
    for i in range(len(self.SP_totalCost)))
212
213         self.RMP.setObjective(obj_master_phase_2, GRB.MINIMIZE)
214
215         # fix the value of artificial variable to 0
216         self.var_Artificial.lb = 0
217         self.var_Artificial.ub = 0
218
219         # solve RMP in Phase 2
220         self.RMP.optimize()
221
222         # update dual variables
223         for i in range(self.NumOrg):
224             for j in range(self.NumDes):
225                 self.dual_CapacityCons[i][j] = self.CapacityCons[i][j].pi
226
227         self.dual_convexCons = self.convexCons.pi
228
229         # update objective of subproblem by dual variables of RMP
230         obj_sub_phase_2 = quicksum((self.Cost[i][j][k] - self.dual_
    CapacityCons[i][j]) * self.var_x[i][j][k] \
231                                    for i in range(self.NumOrg) \
232                                    for j in range(self.NumDes) \
233                                    for k in range(self.NumProduct)) - self.
    dual_convexCons
234
```

```
235         self.subProblem.setObjective(obj_sub_phase_2, GRB.MINIMIZE)
236
237         # update iteration info
238         self.Iter = self.Iter + 1
239
240     def optimizePhase_2(self):
241         # start Phase 2 of dantzig-wolfe decomposition
242         print(" --------- start Phase 2 optimization ---------  ")
243
244         while True:
245             print("---Iter: ", self.Iter)
246
247             # solve subproblem in Phase 2
248             self.subProblem.optimize()
249
250             if self.subProblem.objval >= -1e-6:
251                 print("--- obj of subProblem reaches 0, no new columns---")
252                 break
253             else:
254                 self.Iter = self.Iter + 1
255
256                 # compute the total cost of subproblem solution
257                 # the total cost is the coefficient of RMP when new column is added
258                 totalCost_subProblem = sum(self.Cost[i][j][k] * self.var_x[i
    ][j][k].x \
259                                            for i in range(self.NumOrg) \
260                                            for j in range(self.NumDes) \
261                                            for k in range(self.NumProduct))
262
263                 # update RMP : add new column into RMP
264                 # 1. creat new column
265                 col = Column()
266                 for i in range(self.NumOrg):
267                     for j in range(self.NumDes):
268                         col.addTerms(sum(self.var_x[i][j][k].x for k in
    range(self.NumProduct)), self.CapacityCons[i][j])
269
270                 col.addTerms(1.0, self.convexCons)
271
272                 # 2. add new columns to RMP
273                 self.var_lambda.append(self.RMP.addVar(lb = 0.0
274                                                        , ub = GRB.INFINITY
```

```
                                                    , obj = totalCost_subProblem
                                                    , vtype = GRB.CONTINUOUS
                                                    , name = "lam_phase2_" + str(self.Iter)
                                                    , column = col))

            # solve RMP of Phase 2
            self.RMP.optimize()

            # update dual variables
            for i in range(self.NumOrg):
                for j in range(self.NumDes):
                    self.dual_CapacityCons[i][j] = self.CapacityCons[i][j].pi

            self.dual_convexCons = self.convexCons.pi

            # update objective of subproblem
            obj_sub_phase_2 = quicksum((self.Cost[i][j][k] - self.dual_CapacityCons[i][j]) * self.var_x[i][j][k] \
                                       for i in range(self.NumOrg) \
                                       for j in range(self.NumDes) \
                                       for k in range(self.NumProduct)) - self.dual_convexCons

            self.subProblem.setObjective(obj_sub_phase_2, GRB.MINIMIZE)

    def optimizeFinalRMP(self):
        # obtain the initial solution according the RMP solution
        opt_x = []
        for i in range(self.NumOrg):
            opt_x_commodity = [0.0] * self.NumDes
            opt_x.append(opt_x_commodity)

        # Dantzig-Wolfe decomposition : solve the final model
        print(" --------- start optimization final RMP ---------  ")

        # update objective for RMP
        for i in range(self.NumOrg):
            for j in range(self.NumDes):
```

```
311                opt_x[i][j] = self.Capacity[i][j] + self.var_Artificial.x -
    self.CapacityCons[i][j].slack
312
313            obj_master_final=quicksum(self.Cost[i][j][k] * self.var_x[i][j][k]\
314                                      for i in range(self.NumOrg) \
315                                      for j in range(self.NumDes) \
316                                      for k in range(self.NumProduct))
317
318            self.subProblem.setObjective(obj_master_final, GRB.MINIMIZE)
319
320            for i in range(self.NumOrg):
321                for j in range(self.NumDes):
322                    self.subProblem.addConstr(quicksum(self.var_x[i][j][k] for k
    in range(self.NumProduct)) == opt_x[i][j])
323
324            # solve
325            self.subProblem.setParam("OutputFlag", 1)
326            self.subProblem.optimize()
327
328        def solveMCTP(self):
329            # initialize the RMP and subproblem
330            self.initializeModel()
331
332            # Dantzig-Wolfe decomposition
333            print(" ------------------------------------------")
334            print("    Dantzig-Wolfe Decomposition starts     ")
335            print(" ------------------------------------------")
336            self.optimizePhase_1()
337
338            self.updateModelPhase_2()
339
340            self.optimizePhase_2()
341
342            self.optimizeFinalRMP()
343            print(" ------------------------------------------")
344            print("    Dantzig-Wolfe Decomposition ends     ")
345            print(" ------------------------------------------")
346
347        def reportSolution(self):
348            # report the optimal solution
349            print(" ------------------  solution info ------------------        "
        )
```

```
        print("Objective: ", self.subProblem.objval)
        print("Solution: ")
        for i in range(self.NumOrg):
            for j in range(self.NumDes):
                for k in range(self.NumProduct):
                    if abs(self.var_x[i][j][k].x) > 0:
                        print("  x[%d, %d, %d] = %10.2f" % (i, j, k, round(
    self.var_x[i][j][k].x, 2)))

if __name__ == "__main__":
    MCTP_instance = MCTP()
    MCTP_instance.readData("MCTP.dat")
    MCTP_instance.solveMCTP()
    MCTP_instance.reportSolution()
```

16.5.5 算例格式说明

算例格式说明如下：

算例格式 multicommodity

```
3        供应商个数
7        客户点个数
3        商品种类数

400  800 200
700 1600 300     # supply[i,j] = s_ij, i 是供应商编号,j是产品编号
800 1800 300

300 500 100
300 750 100
100 400   0
 75 250  50              # demand[i][j] = d_ij 需求点i对产品j的需求
650 950 200
225 850 100
250 500 250

625 625 625 625 625 625 625
625 625 625 625 625 625 625
      #容量capacity[i][j] = u_ij, i是起始点编号,j是目的地编号
625 625 625 625 625 625 625

30 39 41
10 14 15
```

```
 8 11 12
10 14 16
11 16 17
71 82 86
 6  8  8
22 27 29
 7  9  9
10 12 13    #cost[i][j][k] = c_ij^k: 每一行是1个s-t对, 表示3种产品的需求量
 7  9  9
21 26 28                          起点1   终点1   30, 39, 41
82 95 99                          起点1   终点2   10, 14, 15
13 17 18
19 24 26
11 14 14
12 17 17
10 13 13
25 28 31
83 99 104
15 20 20
```

16.5.6 算例运行结果

算例运行结果如下：

Results

```
-----------------------------------------
    Dantzig-Wolfe Decomposition starts
 -----------------------------------------
 --------- start Phase 1 optimization ---------
Iter:  0
Iter:  1
Iter:  2
Iter:  3
Iter:  4
---obj of phase 1 reaches 0, phase 1 ends---
 ----start update model of Phase 2 ----
 --------- start Phase 2 optimization ---------
---Iter:  6
---Iter:  7
---Iter:  8
---Iter:  9
---Iter:  10
```

```
---Iter:  11
---Iter:  12
---Iter:  13
---Iter:  14
---Iter:  15
---Iter:  16
---Iter:  17
---Iter:  18
---Iter:  19
---Iter:  20
---Iter:  21
---Iter:  22
---Iter:  23
---Iter:  24
---Iter:  25
---Iter:  26
--- obj of subProblem reaches 0, no new columns---
 --------- start optimization final RMP ---------
Changed value of parameter OutputFlag to 1
   Prev: 0  Min: 0  Max: 1  Default: 1
Gurobi Optimizer version 9.0.1 build v9.0.1rc0 (win64)
Optimize a model with 51 rows, 63 columns and 189 nonzeros
Coefficient statistics:
  Matrix range     [1e+00, 1e+00]
  Objective range  [6e+00, 1e+02]
  Bounds range     [0e+00, 0e+00]
  RHS range        [5e+01, 2e+03]
Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0   -1.4000000e+31   2.400000e+31   1.400000e+01      0s
      20    1.9950000e+05   0.000000e+00   0.000000e+00      0s

Solved in 20 iterations and 0.01 seconds
Optimal objective  1.995000000e+05
 ----------------------------------------
    Dantzig-Wolfe Decomposition ends
 ----------------------------------------
 ------------------  solution info ------------------
Objective:  199500.00000000003
Solution:
  x[0, 4, 0] =      400.00
  x[0, 4, 1] =       75.00
  x[0, 4, 2] =      150.00
```

```
x[0, 5, 1] =     625.00
x[0, 6, 1] =     100.00
x[0, 6, 2] =      50.00
x[1, 0, 0] =     275.00
x[1, 1, 1] =     525.00
x[1, 1, 2] =     100.00
x[1, 2, 1] =     400.00
x[1, 3, 1] =     234.87
x[1, 3, 2] =      50.00
x[1, 4, 0] =     250.00
x[1, 4, 1] =     250.00
x[1, 4, 2] =      50.00
x[1, 5, 1] =     140.13
x[1, 5, 2] =     100.00
x[1, 6, 0] =     175.00
x[1, 6, 1] =      50.00
x[2, 0, 0] =      25.00
x[2, 0, 1] =     500.00
x[2, 0, 2] =     100.00
x[2, 1, 0] =     300.00
x[2, 1, 1] =     225.00
x[2, 2, 0] =     100.00
x[2, 3, 0] =      75.00
x[2, 3, 1] =      15.13
x[2, 4, 1] =     625.00
x[2, 5, 0] =     225.00
x[2, 5, 1] =      84.87
x[2, 6, 0] =      75.00
x[2, 6, 1] =     350.00
x[2, 6, 2] =     200.00
```

16.6 Dantzig-Wolfe 分解求解 VRPTW

本节介绍 Dantzig-Wolfe 分解算法求解车辆路径规划问题（Vehicle Routing Problem，VRP），为了跟之前的章节统一，我们仍以 VRPTW 为例来讨论，具体细节参考文献 (Feillet，2010 和 Desaulniers et al., 2006)。

首先来看 VRPTW 的一般模型，即

$$\min \quad \sum_{k\in K}\sum_{i\in V}\sum_{j\in V} c_{ij}x_{ijk} \tag{16.98}$$

$$\text{s.t.} \quad \sum_{k\in K}\sum_{j\in V} x_{ijk} = 1, \qquad \forall i \in C \tag{16.99}$$

$$\sum_{j\in V} x_{0jk} = 1, \qquad \forall k \in K \tag{16.100}$$

$$\sum_{i\in V} x_{ihk} - \sum_{j\in V} x_{hjk} = 0, \qquad \forall h \in C, \forall k \in K \tag{16.101}$$

$$\sum_{i\in V} x_{i,n+1,k} = 1, \qquad \forall k \in K \tag{16.102}$$

$$\sum_{i\in C} q_i \sum_{j\in V} x_{ijk} \leqslant Q, \qquad \forall k \in K \tag{16.103}$$

$$s_{ik} + t_{ij} - M(1 - x_{ijk}) \leqslant s_{jk}, \qquad \forall (i,j) \in A, \forall k \in K \tag{16.104}$$

$$e_i \leqslant s_{ik} \leqslant l_i, \qquad \forall i \in V, \forall k \in K \tag{16.105}$$

$$x_{ijk} \in \{0,1\}, \qquad \forall (i,j) \in A, \forall k \in K \tag{16.106}$$

观察 VRPTW 的模型，发现约束 (16.100)~(16.105) 都是对每辆车的单独约束，也就是约束都是针对 $\forall k \in K$ 的，只有约束 (16.99) 是对所有车辆 K 进行求和。因此，如果不考虑约束 (16.99)，VRPTW 的模型其实是区块结构的，即每一辆车 k 都是独立的，是一个单独的子问题。而约束 (16.99) 将不同的车辆联系起来。这正好符合 Dantzig-Wolfe 分解的条件。这里可以将约束 (16.99) 作为中央约束（central constraints），然后将约束 (16.100)~(16.105) 按照车辆编号 k 分解成为 K 个独立的子问题。

其中，每个子问题 $k \in K$ 的模型就是一个考虑资源约束（时间窗、容量）的最短路问题。由于 VRPTW 中考虑所有的车辆属性（容量等）都是相同的（也就是同质的车队），因此如果我们固定车辆的编号 k，则子问题的决策变量 x_{ijk} 的下标 k 就可以省去，简化为 x_{ij}，所以子问题的模型可以写成

$$\min \quad \sum_{i\in V}\sum_{j\in V} c_{ij}x_{ij} \tag{16.107}$$

$$\text{s.t.} \quad \sum_{j\in V} x_{0j} = 1, \tag{16.108}$$

$$\sum_{i\in V} x_{ih} - \sum_{j\in V} x_{hj} = 0, \qquad \forall h \in C \tag{16.109}$$

$$\sum_{i\in V} x_{i,n+1} = 1, \tag{16.110}$$

$$\sum_{i\in C} q_i \sum_{j\in V} x_{ij} \leqslant Q, \tag{16.111}$$

$$s_{ik} + t_{ij} - M(1 - x_{ij}) \leqslant s_j, \qquad \forall (i,j) \in A \tag{16.112}$$

$$e_i \leqslant s_i \leqslant l_i, \qquad \forall i \in V \tag{16.113}$$

$$x_{ijk} \in \{0,1\}, \qquad \forall (i,j) \in A \tag{16.114}$$

根据本章前面的介绍，Dantzig-Wolfe 分解中，如果能将每个子问题的所有极点都穷举出来，我们就可以直接将主问题的模型完整地写出来，然后求解主问题，就可以得到最

优解。

注意:子问题的每一个极点,实际上就对应子问题的一个基可行解。但是因为在 VRPTW 中，子问题也是 MIP，所以其极点应该是子问题的 MIP 模型的所有整数可行解的凸包的顶点。

但是在很多情况下，想要穷举出子问题的所有极点是不现实的，因此我们需要用列生成的方式来生成这些极点。每次都求解一个子问题，求得的解就是一个极点。当生成一个极点（基可行解）以后，就会对应的在主问题中产生一个新列，相当于在主问题中添加了一个新的决策变量 θ_i。这个决策变量 θ_i 就表示第 i 个极点对最优解贡献的权重（weight）。并且，在将此列加入主问题的同时，还需要计算 θ_i 对应的这一列（或者这个基可行解）对目标函数的贡献度（即目标函数系数）是多少。

与列生成算法对应章节的介绍一样，我们用 Ω 表示所有子问题的所有基可行解的集合（即所有车辆的所有可行路径集合）。用 Ω' 表示用列生成算法生成的所有基可行解的集合。用 x_{ij}^k 表示第 k 辆车对应的子问题的基可行解，用 θ_u^k 表示第 k 辆车产生的第 u 个基可行解（极点）对最优解贡献的权重系数，也就是路径 $r_u^k \in \Omega'$ 在最优解中对应的系数（被选择的比例）。因此 VRPTW 的 Dantzig-Wolfe 分解方法的主问题（Master Problem）可以写成

$$\min \sum_{k\in K}\sum_{(i,j)\in A} c_{ij}\left(\sum_{r_k^u\in\Omega'} x_{ijk}\theta_k^u\right) \tag{16.115}$$

$$\text{s.t.} \quad \sum_{k\in K}\sum_{(i,j)\in A} c_{ij}\left(\sum_{r_k^u\in\Omega'} x_{ijk}\theta_k^u\right) \geqslant 1, \quad \forall i \in V\backslash\{0\} \tag{16.116}$$

$$\sum_{r_k^u\in\Omega'} \theta_k^u = 1, \qquad \forall k \in K \tag{16.117}$$

$$\theta_k^u \in \{0,1\}, \qquad \forall r_k^u \in \Omega', \forall k \in K \tag{16.118}$$

子问题的每一个基可行解（极点）都对应一条完整的路径，因此这一列对目标函数的贡献度就是这条路径的长度，因此为 $\sum_{(i,j)\in A} c_{ij}x_{ijk}$，其中 x_{ijk} 的下标 k 其实也可以没有，只是为了标识这是第 k 个子问题的解。因此该列在目标函数中产生的项就是 $\theta_k^u \cdot \sum_{(i,j)\in A} c_{ij}x_{ijk}$，实际上就是选择该条路径会增加多少行驶距离。

在中央约束 (16.116) 中，该条路径的贡献度即为这条路径访问点 i 的次数（这里我们固定点 i），因此在中央约束中，决策变量 θ_k^u 的约束系数为 $\sum_{(i,j)\in A} x_{ijk}$。

与列生成算法相应章节的做法相同，我们令

$$a_{ik} = \sum_{(i,j)\in A} x_{ijk}, \quad \forall i \in V\backslash\{0\} \tag{16.119}$$

$$c_k^u = \sum_{(i,j)\in A} c_{ij}x_{ijk} \tag{16.120}$$

则限制性主问题可以改写成下面的集分割（Set Partitioning）模型：

$$\min \quad \sum_{r_k \in \Omega'} c_k \theta_k \tag{16.121}$$

$$\text{s.t.} \quad \sum_{r_k \in \Omega'} a_{ik} \theta_k \geqslant 1, \quad \forall i \in V \backslash \{0\} \tag{16.122}$$

$$\sum_{r_k \in \Omega'} \theta_k \leqslant K \tag{16.123}$$

$$\theta_k \in \mathbb{N}, \qquad \forall r_k \in \Omega' \tag{16.124}$$

注意：因为所有车辆都是同质的，所以所有车辆的子问题都是相同的，我们也不区分产生的路径具体来自哪辆车，直接用下标 k 表示 Ω' 中第 k 条路径。

容易观察到，上述模型和列生成算法求解 VRPTW 的情况是完全相同的。这也是一点比较重要的发现，用 Dantzig-Wolfe 分解求解 VRPTW 和用列生成算法求解 VRPTW 本质上是相同的，因此对于 Dantzig-Wolfe 分解求解 VRPTW 我们不做专门的实现，读者可以参考列生成算法求解 VRPTW 部分的代码即可。

第17章 Benders分解算法

Benders 分解算法是另外一种强大的算法，该算法由 Benders J. F. 于 1962 年首次提出。区别于列生成算法不断地添加新列，Benders 分解不断地添加新的行，是一种 Row Generation（行生成）的方法。当然，在一些鲁棒优化问题中（两阶段鲁棒优化），还会有同时使用列生成和行生成的方法，叫作列与约束生成（Column and Constraint Generation，C&CG）。掌握了本章的读者，可以继续深入研究该算法。本章就来详细地介绍 Benders 分解算法。我们首先介绍 Benders 分解的基本原理，然后以一个非常详细的例子来帮助大家理解该算法的完整过程。本章内容主要参考文献（Taşkin，2011 和 Kalvelagen，2002）。

17.1 分解方法

17.1.1 Benders 分解的原理

考虑如下的问题：

$$\min_{\boldsymbol{x},\boldsymbol{y}} \quad \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}+\boldsymbol{f}^{\mathrm{T}}\boldsymbol{y} \tag{17.1}$$

$$\text{s.t.} \quad \boldsymbol{A}\boldsymbol{x}+\boldsymbol{B}\boldsymbol{y}=\boldsymbol{b} \tag{17.2}$$

$$\boldsymbol{x}\geqslant \boldsymbol{0} \tag{17.3}$$

$$\boldsymbol{y}\in Y\subseteq \mathbb{R}^{q} \tag{17.4}$$

其中，$\boldsymbol{x},\boldsymbol{y}$ 分别是 $p\times 1, q\times 1$ 维的决策变量。Y 是一个多面体形状的可行域，$\boldsymbol{A},\boldsymbol{B}$ 是矩阵，并且 $\boldsymbol{b},\boldsymbol{c},\boldsymbol{f}$ 是相应维度的向量。

假设变量 $\boldsymbol{y}$ 是复杂决策变量（比如整数决策变量），如果 $\boldsymbol{y}$ 被固定之后，也许由于矩阵 $\boldsymbol{A}$ 特殊的结构，问题会变得更容易求解，比如，若矩阵 $\boldsymbol{A}$ 是 unimodular 的矩阵（幺模矩阵，见第 2.4 节），则剩余部分的问题就具有整数最优解特性，直接可以松弛成 LP 求解（如果 $\boldsymbol{x}$ 是整数变量）；或者说，$\boldsymbol{y}$ 是整数变量，$\boldsymbol{x}$ 是连续变量，$\boldsymbol{y}$ 固定以后，剩余部分就变成了 LP，就可以很容易求解，从而可以通过这种思想，将原来非常复杂的问题变得更容易求解。

Benders 分解就是基于上述思想的一种分解算法。Benders 分解是将问题 (17.1) 分解成两个问题：

（1）一个主问题：包含变量 $\boldsymbol{y}$；

（2）一个子问题：包含变量 $\boldsymbol{x}$。

这 2 个问题各自求解，就比之前直接求解 (17.1) 容易很多。

首先，我们将问题 (17.1) 写成如下的等价形式：

$$\min_{y\in Y} \quad \boldsymbol{f}^{\mathrm{T}}\boldsymbol{y}+q(\boldsymbol{y}) \tag{Master problem}$$

$$\text{s.t.} \quad \boldsymbol{y}\in Y\subseteq\mathbb{R}^{q} \tag{17.5}$$

其中，$q(\boldsymbol{y})$ 就是我们所要构建的子问题的目标函数，其定义如下：

$$q(\boldsymbol{y})=\min_{\boldsymbol{x}\geqslant 0} \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \tag{Subproblem}$$

$$\text{s.t.} \quad \boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}} \tag{17.6}$$

$q(\boldsymbol{y})$ 就是给定了 $\boldsymbol{y}$ 以后产生的子问题 (Subproblem) 的目标函数。对于子问题来讲，$\boldsymbol{y}$ 就是一个输入参数，是一个已知的值。

当然，上述主问题 (Master problem) 和子问题也可以简单地写成

$$\min_{\boldsymbol{y}\in Y}\left[\boldsymbol{f}^{\mathrm{T}}\boldsymbol{y}+\min_{\boldsymbol{x}\geqslant 0}\left\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}\mid \boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y}\right\}\right] \tag{17.7}$$

注意到，如果存在某个 $\boldsymbol{y}$ 使得子问题无界，则可知主问题也无界，因为 $q(\boldsymbol{y})$ 无界。这就意味着原问题 (17.1)-(17.4) 也无界。

假设子问题有界，我们可以通过求解子问题来计算出 $q(\boldsymbol{y})$。在本章的代码中我们也是这么做的。这里为了阐述算法原理，先不直接求解子问题，而是取子问题的对偶，通过求解子问题的对偶问题来计算出 $q(\boldsymbol{y})$。用 $\boldsymbol{\alpha}$ 表示约束 (17.6) 的对偶变量，则子问题的对偶问题可以写成

$$\max_{\boldsymbol{\alpha}} \quad (\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})^{\mathrm{T}}\boldsymbol{\alpha} \tag{Subproblem-Dual}$$

$$\text{s.t.} \quad \boldsymbol{A}^{\mathrm{T}}\boldsymbol{\alpha}\leqslant c \tag{17.8}$$

$$\boldsymbol{\alpha}\ \text{free} \tag{17.9}$$

这里很多初学者会疑惑，为什么要取对偶？原因是这样的，我们观察对偶问题 (Subproblem-Dual) 的约束，发现该约束与 $\bar{\boldsymbol{y}}$ 无关，这样就更容易处理，每次迭代中，不需要更新子问题的所有约束，只需要更新子问题的目标函数即可。其次，取对偶更重要的作用在于对偶问题可以帮助我们推出一些重要的结论。具体见下文介绍。我们通过提取约束 (17.8) 的对偶变量，就可以得到原来子问题的决策变量 $\boldsymbol{x}$ 的取值。

如果原模型 (17.1) 中的约束是 $\boldsymbol{A}\boldsymbol{x}+\boldsymbol{B}\boldsymbol{y}\geqslant\boldsymbol{b}$，则对偶问题中的对偶变量的取值范围约束 (17.9) 就变成 $\boldsymbol{\alpha}\geqslant\boldsymbol{0}$。

上文介绍到，通过观察可以发现一个关键的现象，决策变量 $\boldsymbol{y}$ 并不影响子问题的对偶问题的可行域，而只影响对偶问题的目标函数。因此，如果对偶问题的可行域为空，则必定会出现下面两种情况之一（对偶问题不可行，原问题要么无界，要么不可行）：

（1）存在某个 $\boldsymbol{y}\in Y$，使得对偶问题的原问题 (Subproblem) 无界（在 $\boldsymbol{y}$ 的这种取值下，初始问题 (17.1) 也是无界的）；

（2）对于所有的 $\boldsymbol{y} \in Y$，子问题 (Subproblem) 均不可行，此时初始问题 (17.1) 也是不可行的。这里为什么变成了对所有的 $\boldsymbol{y}$ 呢？因为子问题的对偶问题 (Subproblem-Dual) 的可行性与 $\boldsymbol{y}$ 的取值无关。所以只要至少存在一个 $\bar{\boldsymbol{y}}$ 使得子问题的对偶问题不可行，则不论 $\boldsymbol{y}$ 取其他任何值，对偶问题也均不可行。实际上，如果不论 $\boldsymbol{y}$ 取什么值都会导致对偶问题不可行，且子问题也均不可行，则说明不存在 $\boldsymbol{y}$ 使得子问题可行，容易得出，初始问题 (17.1) 本身不可行。

以上两种情况是根据对偶定理得到的，即原问题（对偶问题）无可行解，对偶问题（原问题）无界或无可行解。这里我们需要一些对偶理论方面的知识来帮助理解上面的内容，具体参见第 4.3 节。

假设对偶问题 (Subproblem-Dual) 的可行域是非空的，非空的可行域就是由一系列极点（extreme point）或者极射线（extreme ray）构成的。因此，可以穷举出问题 (Subproblem-Dual) 的可行域的所有的极点 $\left(\boldsymbol{\alpha}_p^1, \cdots, \boldsymbol{\alpha}_p^I\right)$ 和所有的极射线 $\left(\boldsymbol{\alpha}_r^1, \cdots, \boldsymbol{\alpha}_r^J\right)$，其中，$I$ 和 J 是极点和极射线的个数。对偶问题 (Subproblem-Dual) 可以通过以下方法来等价地求解（是一种等价的求解方法，目的是将对偶问题 (Subproblem-Dual) 中的 max 变成 min 问题，从而实现主问题与子问题的交互）。

给定向量 $\bar{\boldsymbol{y}}$，可以通过检查如下事项来求解子问题的对偶问题：

（1）是否存在一条极射线 $\boldsymbol{\alpha}_r^j$ 使得子问题的对偶问题无界。根据对偶定理可得，此时子问题无可行解。对偶问题无界，等价于是否存在极射线 $\boldsymbol{\alpha}_r^j$，使得 $(\boldsymbol{\alpha}_r^j)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})>0$ (实际上，这是一个定理，这里可以不加证明地使用这个定理)。若存在，因为对偶问题是 max 问题，所以目标函数就会取到无穷大，进而导致问题无界。对偶问题无界，则子问题的原问题就无可行解。综上，如果对偶问题有有界的可行解，则它必须满足对所有的极射线 $\boldsymbol{\alpha}_r^j$ 都有 $(\boldsymbol{\alpha}_r^j)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}}) \leqslant 0$。因此，这条约束应该被加入重构的对偶问题中。

> 这里需要进一步解释一下，本部分讨论的前提假设是：初始问题 (17.1) 存在有界的可行解 (下同)。基于此前提，如果存在极射线 $\boldsymbol{\alpha}_r^j$ 使得 $(\boldsymbol{\alpha}_r^j)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})>0$，从而使得子问题的对偶问题无界，子问题无可行解，这说明主问题提供的 $\bar{\boldsymbol{y}}$ 不合适。这个不合适的 $\bar{\boldsymbol{y}}$ 使得子问题无可行解。为了让主问题在任何一步迭代中都不要产生使子问题不可行的 $\bar{\boldsymbol{y}}$，可以通过遍历所有可行的 $\bar{\boldsymbol{y}}$，找到所有对应的极射线 $\boldsymbol{\alpha}_r^j$，如果这些 $\bar{\boldsymbol{y}}$ 对应的子问题均有可行解 (也就是子问题的对偶问题一定有界)，则这些极射线必然均满足 $(\boldsymbol{\alpha}_r^j)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}}) \leqslant 0$。因为对所有可行的 $\bar{\boldsymbol{y}}$，都满足上述表达式，所以可以直接将 $\bar{\boldsymbol{y}}$ 替换成 $\boldsymbol{y}$。因此，加回到主问题的割平面形式就是 $(\boldsymbol{\alpha}_r^j)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y}) \leqslant 0$。
>
> 但是，初始化的主问题中可行的 $\bar{\boldsymbol{y}}$ 非常多，对应的极射线 $\boldsymbol{\alpha}_r^j$ 以及割平面也非常多，一开始就将它们全部识别并加入是无法处理的。不过可以通过迭代，逐步将其识别并加入到主问题中。
>
> 由于上述割平面是为了帮助主问题得到使子问题可行的 $\bar{\boldsymbol{y}}$，排除那些导致子问题不可行的 $\bar{\boldsymbol{y}}$，这相当于找到了分解后子问题可行的必要条件，所以这个割平面也被称

为 Benders feasibility cut。

（2）找到一个极点 α_p^i，使得目标函数 $(\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$ 取得有界的最大值。此时，子问题的原问题和对偶问题都有有限个有界的最优解。我们从这有限个解中筛选出目标值最大的解（当然了，多个最优解的目标函数肯定是相同的，不过这不影响上面说法的正确性），就相当于求解了子问题的对偶问题。具体来讲，可以令子问题的对偶问题的最优目标函数值为 q，则对每个极点 α_p^i，q 必然满足 $q \geqslant (\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$，然后设置目标函数为 min q，就可以得到最优目标函数值 q，达到成功求解子问题的对偶问题的目的。

因为约束 $q \geqslant (\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$ 和目标函数 min q 共同作用后，可以达到同时找到子问题的对偶问题和子问题的最优目标值的作用，所以，为了将子问题的解对主问题产生的影响逐渐添加回去（因为一开始是直接将子问题对主问题的影响全部忽略掉），我们将识别子问题的最优值的约束加回到主问题中，具体来讲，就是将割平面 $q \geqslant (\alpha_p^i)^{\mathrm{T}}(b-By)$ 加回到主问题中。注意，加回主问题时，y 不再是固定后的 $\bar{y}$，而是变成了决策变量 y。这个割平面的意思是，对于给定的 $\bar{y}$，我们通过子问题的对偶问题，得到了一个可以最大化 $(\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$ 的极点 α_p^i，该极点可以帮助我们得出基于 $\bar{y}$ 的子问题的最优性的必要条件（其实就是识别出子问题的最优目标值），并且可以识别主问题中决策变量 q 的取值的一个下界，即 $(\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$（该下界是确保不会剔除最优解的）。因为该下界对应的割平面是基于子问题的最优性条件得来的，所以被称为 Benders optimality cut。这里可以再做一些直观分析，在刚开始时，主问题中 q 的取值一定为 0，随着 Benders optimality cut(也就是下界) 的不断加入，主问题中 q 的取值逐渐上升，最终得到最优解对应的 q 值。这是一种追索 (recourse)，有一些欲擒故纵的味道。具体来讲，就是先忽略一部分约束的影响，然后在后期的迭代中“秋后算账”，逐渐将子问题的约束对主问题的影响加回来。

接下来，我们再从弱对偶性的角度提供另一种更好的解释，帮助读者进一步理解 Benders optimality cut。记 $q(\bar{y})=c^{\mathrm{T}}x$ 为基于主问题提供的 $\bar{y}$ 构建的子问题的目标函数。根据弱对偶性，如果子问题和子问题的对偶问题都有有界的可行解，则一定满足 $q(\bar{y})=c^{\mathrm{T}}x \geqslant (\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$。这表明，给定 $\bar{y}$，原始问题 (17.1) 的目标函数中 $c^{\mathrm{T}}x$ 的部分的下界是 $(\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$。如果我们不断改变 $\bar{y}$，则在每个 $\bar{y}$ 处，都必然满足 $q(\bar{y})=c^{\mathrm{T}}x \geqslant (\alpha_p^i)^{\mathrm{T}}(b-B\bar{y})$，其中 α_p^i 是在对应的 $y=\bar{y}$ 时，子问题的对偶问题的极点。如果我们能够一次性将每个可行解 $\bar{y}$ 对应的 α_p^i 都找到，并构造割平面加到主问题中，则可以直接得到原始问题 (17.1) 的等价形式。但是可行的 $\bar{y}$ 及其对应的极点 α_p^i 和割平面非常多，最终在主问题中真正 binding 的割平面也往往只有一部分。所以我们可以通过迭代的方式，将可能会 binding 的割平面找出来，加到主问题中，这也正是 Benders 分解算法要做的事情。基于上述解释，对于所有的 y，表达式

$(\boldsymbol{\alpha}_p^i)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})$ 都为 $\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}$ 提供了一个下界，即 $q(\boldsymbol{y})=\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}\geqslant(\boldsymbol{\alpha}_p^i)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})$。因此，在将该割平面加回到主问题时，可以直接使用 $\boldsymbol{y}$，而不再使用 $\bar{\boldsymbol{y}}$，所以加回主问题的割平面表达式为 $q\geqslant(\boldsymbol{\alpha}_p^i)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})$。

此外，如果在子问题中引入一组辅助约束 $\boldsymbol{y}=\bar{\boldsymbol{y}}$，同时，使用拉格朗日松弛方法，构造相应的拉格朗日对偶问题并求解，则可以得到 Benders optimality cut 的另外一种形式，即 $q\geqslant\boldsymbol{c}^{\mathrm{T}}\bar{\boldsymbol{x}}+\boldsymbol{\pi}^{\mathrm{T}}(\boldsymbol{y}-\bar{\boldsymbol{y}})$。其中 $\bar{\boldsymbol{y}}$ 即为主问题向子问题提供的 $\boldsymbol{y}$ 的取值，$\bar{\boldsymbol{x}}$ 是直接求解子问题本身 (Subproblem) 得到的解，$\boldsymbol{\pi}$ 为子问题中约束 $\boldsymbol{y}=\bar{\boldsymbol{y}}$ 的对偶变量。这种方法也叫 Benders 对偶分解。

基于此，子问题的对偶问题 (Subproblem-Dual) 可以等价地写成如下形式：

$$\begin{aligned}
\min\quad & q && \text{(Subproblem-Dual-Reformulated)}\\
\text{s.t.}\quad & \left(\boldsymbol{\alpha}_r^j\right)^{\mathrm{T}}\left(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}}\right)\leqslant 0, && \forall j=1,2,\cdots,J && (17.10)\\
& \left(\boldsymbol{\alpha}_p^i\right)^{\mathrm{T}}\left(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}}\right)\leqslant q, && \forall i=1,2,\cdots,I && (17.11)\\
& q \text{ free} && && (17.12)
\end{aligned}$$

经过如上操作，子问题由原问题 (Subproblem) 的 min 变成了对偶问题 (Subproblem-Dual) 的 max，而这个 max 的对偶问题又等价于问题 Subproblem-Dual-Reformulated的 min 的形式。经过两步等价变换，Master Problem 和 Subproblem 的目标函数优化方向就达成了统一，都是 min，这样原问题和子问题就可以无缝交互，顺畅地使用 Benders 分解了。当然，这些转化是给出原问题和子问题将如何交互以及这样操作的原理。但是，在实际的代码实现中，也可以通过直接求解子问题本身，通过求解器提供的相关函数，直接获得极点和极射线，并构成对应的 Benders optimality cut 和 Benders feasibility cut，传给原问题。

注意到模型 Subproblem-Dual-Reformulated中有一个单独的变量 q，以及非常多的约束（$I+J$ 个约束）。我们将主问题 (Master problem) 中的第二项 $q(\boldsymbol{y})$ 直接用 Subproblem-Dual-Reformulated进行替换，就得到了原问题 (17.1) 或者 Master problem的等价形式（含有变量 $\boldsymbol{y}$ 和 q）。

因此，我们可以写出原问题（Original Problem）的等价形式：

$$\begin{aligned}
\min\quad & \boldsymbol{f}^{\mathrm{T}}\boldsymbol{y}+q && \text{(Benders Original Problem Reformulated)}\\
\text{s.t.}\quad & \left(\boldsymbol{\alpha}_r^j\right)^{\mathrm{T}}\left(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y}\right)\leqslant 0, && \forall j=1,2,\cdots,J && (17.13)\\
& \left(\boldsymbol{\alpha}_p^i\right)^{\mathrm{T}}\left(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y}\right)\leqslant q && \forall i=1,2,\cdots,I && (17.14)\\
& \boldsymbol{y}\in Y,\quad q \text{ free} && && (17.15)
\end{aligned}$$

由于问题 Subproblem-Dual的极点和极射线是随着问题规模呈指数级增长的，直接尝试穷举子问题中的所有极点和极射线去构造所有的 Benders optimality cut 和 Benders feasibility cut 是不可行的做法。因此，Benders 分解首先使用这些 Cuts 里面的一个子集（也

就是一部分 Cuts)，并且求解“松弛的主问题”(因为松弛掉了一部分约束)，这会产生一个候选最优解 (y^*, q^*)。然后，我们求解子问题的对偶问题 (Subproblem-Dual)，计算出 $q(y^*)$ (此时，问题 (Subproblem-Dual) 中的 y 被固定成了 y^*)。如果子问题的最优解 $q(y^*) = q^*$，则算法停止，得到了最优解。否则，如果对偶子问题 (Subproblem-Dual) 是无界的，则构造割平面 (17.13)，并加入松弛的主问题 (Benders Original Problem Reformulated) 中去，然后再求解主问题 (Benders Original Problem Reformulated)。其中割平面 (17.13) 指的就是 Benders feasibility cuts，因为它们是问题可行的必要条件。

类似地，如果子问题有最优解 $q(y^*) > q^*$，则构造割平面 (17.14) 并将其加入到主问题 (Benders Original Problem Reformulated) 中，然后再求解新的主问题 (Benders Original Problem Reformulated) 。(约束 (17.14) 被称为 Benders optimality cuts，因为它们是主问题达到最优的必要条件) 由于 I 和 J 是有限的，并且新的 feasibility cuts 和 optimality cuts 在每次迭代中都会被生成并添加到主问题中，因此，该问题会在有限次迭代之后得到最优解。

17.1.2 Benders 分解的全过程

整个 Benders 分解的流程如下。

(1) 首先求解初始主问题

$$\min_{y \in Y} \quad f^{\mathrm{T}} y + q(y) \qquad \text{(Master problem)}$$

$$\text{s.t.} \quad y \in Y \subseteq \mathbb{R}^q \qquad (17.16)$$

这样会得到最优解。最优解中 y 的值 $\bar{y}$ 当作固定值代入下一步的迭代中。

(2) 将 Master problem中的最优解 $\bar{y}$ 代入子问题对偶问题 (Subproblem-Dual) 中，求解 (Subproblem-Dual)

$$\max_{\alpha} \quad (b - B\bar{y})^{\mathrm{T}} \alpha \qquad \text{(Subproblem-Dual)}$$

$$\text{s.t.} \quad A^{\mathrm{T}} \alpha \leqslant c \qquad (17.17)$$

$$\alpha \text{ free} \qquad (17.18)$$

得到 (Subproblem-Dual) 的极点和极射线。如果有最优解，就获得其目标值 $q(y^*) = \alpha^{\mathrm{T}^*}(b - B\bar{y})$。(因为 (Subproblem) 和 (Subproblem-Dual) 互为对偶，故目标值相同。)

(3) 求解完子问题的对偶问题以后，将极点和极射线的相关约束加入松弛的主问题中，更新主问题，主问题变成等价的形式 (Benders Original Problem Reformulated)，求解更新后的模型：

$$\min \quad f^{\mathrm{T}} y + q$$

$$\text{s.t.} \quad \left(\alpha_r^j\right)^{\mathrm{T}} (b - By) \leqslant 0, \quad \forall j = 1, 2, \cdots, J \qquad (17.19)$$

$$\left(\alpha_p^i\right)^{\mathrm{T}} (b - By) \leqslant q, \quad \forall i = 1, 2, \cdots, I \qquad (17.20)$$

$$y \in Y, \quad q \text{ free} \qquad (17.21)$$

获得目标值中 q 的值 q^*。直到 $q(y^*)=q^*$，算法停止，得到最优解。这里的终止条件也可从全局 UB 和全局 LB 得到。Benders 分解中，$f^{\mathrm{T}}y+q$ 提供了一个全局 LB，$f^{\mathrm{T}}y+q(y)$ 提供了一个全局 UB。因此，为 UB=LB 时，有 $q(y)=q$。

17.1.3 算法框架图

Benders 分解算法框架图如图 17.1 所示。

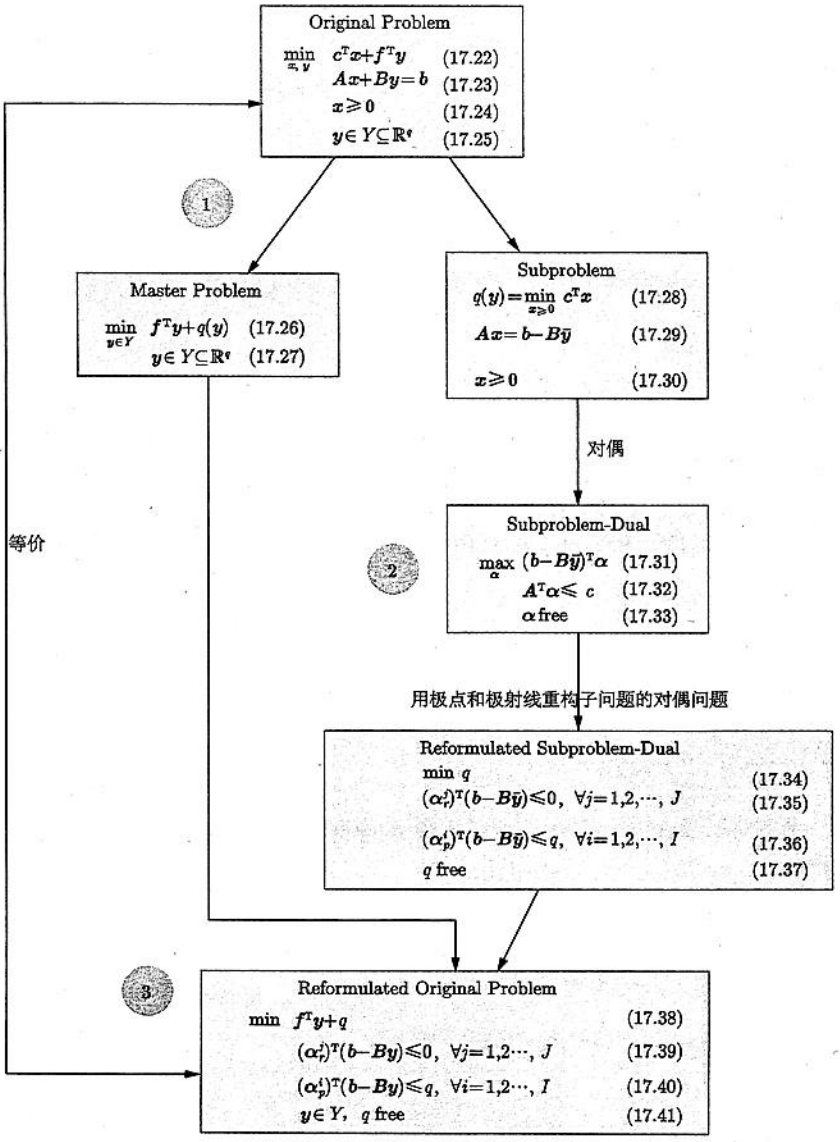


图 17.1 Benders 分解算法框架图

根据算法框图，Benders 分解算法迭代的过程如下。

（1）首先初始化模型①，且令 $q(\boldsymbol{y})=q$。初始化后的①就等价于初始化的③，然后求解模型③，该模型松弛掉了所有关于简单变量 $\boldsymbol{x}$ 的约束（因为 $\boldsymbol{x}$ 是子问题的变量），仅留下复杂变量 $\boldsymbol{y}$ 以及只跟复杂变量 $\boldsymbol{y}$ 这一组变量相关的约束。求初始化后的模型③我们可以得到固定的 $\bar{\boldsymbol{y}}$，以及 q 的值 $\bar{q}$。(注意：如果确定 $q=\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}$ 这一项在最优解中的取值为正的话，也可以对应地在初始化的③中添加一个非负约束：$q\geqslant 0$。)

（2）求解②或者求解②的原问题，如果②无界，则得到该问题的极射线；如果②有最优解，则获取该问题的极点（极点坐标）。

（3）根据获得的极射线或者极点，构造 Benders feasibility cut 和 Benders optimality cut。并将极点和极射线产生的 Benders feasibility cut 和 Benders optimality cut 加入模型③，然后求解模型③。

（4）如果前一步求解模型③得到的 $\bar{q}$ 的值与当前迭代步骤中求解模型②得到的目标函数 $q(\bar{\boldsymbol{y}})=(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})^{\mathrm{T}}\boldsymbol{\alpha}$ 相等，则得到最优解。否则将③求得的解代入模型②，循环步骤（2），（3）和（4）。

17.1.4 算法伪代码

Benders 分解算法伪代码如下：

Algorithm 20 Benders 分解算法

初始化: $y\leftarrow$ 初始可行整数解
设置 $\epsilon\leftarrow$ 一个小的正数容差 (如 0.001)
$\mathrm{LB}\leftarrow-\infty$
$\mathrm{UB}\leftarrow\infty$
while $\mathrm{UB}-\mathrm{LB}\geqslant\epsilon$ **do**
    (****** Solve Subproblem ******)
    $\min\limits_{\alpha}\left\{\boldsymbol{f}^{\mathrm{T}}\bar{\boldsymbol{y}}+(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})^{\mathrm{T}}\boldsymbol{\alpha}\,|\boldsymbol{A}^{\mathrm{T}}\boldsymbol{\alpha}\leqslant c,\boldsymbol{\alpha}\text{ free}\right\}$
    if Unbounded **then**
        得到极射线 $\bar{\boldsymbol{\alpha}}$
        添加割平面到 $(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})^{\mathrm{T}}\bar{\boldsymbol{\alpha}}\leqslant 0$ 到 master problem 中
    else
        得到极点 $\bar{\boldsymbol{\alpha}}$
        添加割平面 $z\geqslant\boldsymbol{f}^{\mathrm{T}}\boldsymbol{y}+(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})^{\mathrm{T}}\bar{\boldsymbol{\alpha}}$ 到 master problem 中
        $\mathrm{UB}\leftarrow\min\left\{\mathrm{UB},\boldsymbol{f}^{\mathrm{T}}\bar{\boldsymbol{y}}+(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})^{\mathrm{T}}\bar{\boldsymbol{\alpha}}\right\}$
    end if
    (****** Solve Master Problem ******)
    $\min\limits_{y}\ \{z|\text{cuts},\boldsymbol{y}\in Y\}$
    $\mathrm{LB}\leftarrow\bar{z}$
end while

在用 Benders 分解算法求解 MIP 时，如果子问题和主问题都比较容易求解，但是原始问题不好求解，其求解效率是比较高的。

17.2 详细案例

Benders 分解原理比较复杂，不容易完全理解。这里我们以一个非常详细完整的例子，按照 Benders 分解算法的流程执行一遍，以帮助读者更好地理解。本例来自参考文献（Taşkin, 2011）。

17.2.1 问题描述和模型转换

考虑如下的优化问题：

$$\min \quad 7y_1+7y_2+7y_3+7y_4+7y_5+x_1+x_2+x_3+x_4+x_5 \tag{17.22}$$

$$\text{s.t.} \quad x_1+x_4+x_5=8 \tag{17.23}$$

$$x_2+x_5=3 \tag{17.24}$$

$$x_3+x_4=5 \tag{17.25}$$

$$x_1 \leqslant 8y_1 \tag{17.26}$$

$$x_2 \leqslant 3y_2 \tag{17.27}$$

$$x_3 \leqslant 5y_3 \tag{17.28}$$

$$x_4 \leqslant 5y_4 \tag{17.29}$$

$$x_5 \leqslant 5y_5 \tag{17.30}$$

$$x_1,\ x_2,\ x_3,\ x_4,\ x_5 \geqslant 0 \tag{17.31}$$

$$y_1,\ y_2,\ y_3,\ y_4,\ y_5 \in \{0,1\} \tag{17.32}$$

根据 Benders 分解的方法，我们将其分解为主问题和子问题。由于变量 $\boldsymbol{y}$ 是 0-1 变量，相比于 $\boldsymbol{x}$ 是一个较为复杂的变量。因此考虑固定 $\boldsymbol{y}$。

对于给定的 $\bar{\boldsymbol{y}}$ 向量，子问题的原问题可以表示为下面的形式：

$$\min \quad x_1+x_2+x_3+x_4+x_5 \qquad \text{(Subproblem}(\bar{\boldsymbol{y}} \text{ fixed))}$$

$$\text{s.t.} \quad x_1+x_4+x_5=8 \tag{17.33}$$

$$x_2+x_5=3 \tag{17.34}$$

$$x_3+x_4=5 \tag{17.35}$$

$$x_1 \leqslant 8\bar{y}_1 \tag{17.36}$$

$$x_2 \leqslant 3\bar{y}_2 \tag{17.37}$$

$$x_3 \leqslant 5\bar{y}_3 \tag{17.38}$$

$$x_4 \leqslant 5\bar{y}_4 \tag{17.39}$$

$$x_5 \leqslant 5\bar{y}_5 \tag{17.40}$$

$$x_1,\ x_2,\ x_3,\ x_4,\ x_5 \geqslant 0 \tag{17.41}$$

用 $\alpha_1,\alpha_2,\alpha_3$ 分别表示约束 (17.33)~(17.35) 的对偶变量，用 $\beta_1,\beta_2,\beta_3,\beta_4,\beta_5$ 表示约束 (17.36)~(17.40) 的对偶变量，我们得到子问题 (Subproblem($\bar{\boldsymbol{y}}$ fixed)) 的对偶问题如下：

$$\max\ 8\alpha_1+3\alpha_2+5\alpha_3+8\bar{y}_1\beta_1+3\bar{y}_1\beta_2+5\bar{y}_3\beta_3+5\bar{y}_4\beta_4+3\bar{y}_5\beta_5 \quad \text{(Subproblem}(\bar{\boldsymbol{y}}\text{ fixed})\text{-Dual)}$$

$$\text{s.t.}\ \alpha_1 + \beta_1 \leqslant 1 \tag{17.42}$$

$$\alpha_2 + \beta_2 \leqslant 1 \tag{17.43}$$

$$\alpha_3 + \beta_3 \leqslant 1 \tag{17.44}$$

$$\alpha_1 + \alpha_3 + \beta_4 \leqslant 1 \tag{17.45}$$

$$\alpha_1 + \alpha_2 + \beta_5 \leqslant 1 \tag{17.46}$$

$$\alpha_1,\ \alpha_2,\ \alpha_3 \ \text{free} \tag{17.47}$$

$$\beta_1,\ \beta_2,\ \beta_3,\ \beta_4,\ \beta_5 \leqslant 0 \tag{17.48}$$

构建好了上述主问题和子问题，我们就可以开始执行 Benders 分解算法的迭代了。

17.2.2 第 1 次迭代

首先我们初始化主问题。将主问题中的所有的 Benders cuts 都松弛掉，并且令 $q = x_1+x_2+x_3+x_4+x_5$，由于 $\boldsymbol{x}$ 为非负变量，因此 $q \geqslant 0$。我们构造出下面的初始主问题：

$$\min\ 7y_1+7y_2+7y_3+7y_4+7y_5+q \quad \text{(Initial Master Problem Iter 1)}$$

$$\text{s.t.}\ q \geqslant 0 \tag{17.49}$$

$$y_1,\ y_2,\ y_3,\ y_4,\ y_5 \in \{0,1\} \tag{17.50}$$

求解 (Initial Master Problem Iter 1)，问题最优解是

$$\bar{\boldsymbol{y}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \end{bmatrix}^{\mathrm{T}}$$

$$\bar{q} = 0$$

这里我们得到了 $\boldsymbol{y}$ 的值 $\bar{\boldsymbol{y}}$，根据 Benders 分解算法的步骤，我们将子问题中的 $\bar{\boldsymbol{y}}$ 替换成上面的值，也就是固定 $\boldsymbol{y} = \bar{\boldsymbol{y}}$，然后将 $\bar{\boldsymbol{y}}$ 代入子问题 (Subproblem($\bar{\boldsymbol{y}}$ fixed)-Dual)，更新子问题。更新后的 (Subproblem($\bar{\boldsymbol{y}}$ fixed)-Dual) 变化为

$$\max\ 8\alpha_1+3\alpha_2+5\alpha_3+0\ +0\ +0\ +0\ +0 \quad \text{(SP}(\bar{\boldsymbol{y}}\text{ fixed})\text{-Dual iter 1)}$$

$$\text{s.t.}\ \alpha_1 + \beta_1 \leqslant 1 \tag{17.51}$$

$$\alpha_2 \qquad +\beta_2 \qquad \leqslant 1 \tag{17.52}$$

$$\alpha_3 \qquad +\beta_3 \qquad \leqslant 1 \tag{17.53}$$

$$\alpha_1 \quad +\alpha_3 \qquad +\beta_4 \quad \leqslant 1 \tag{17.54}$$

$$\alpha_1 \ +\alpha_2 \qquad +\beta_5 \leqslant 1 \tag{17.55}$$

$$\alpha_1,\ \alpha_2,\ \alpha_3 \qquad \text{free} \tag{17.56}$$

$$\beta_1,\ \beta_2,\ \beta_3,\ \beta_4,\ \beta_5 \leqslant 0 \tag{17.57}$$

求解 (SP($\bar{\boldsymbol{y}}$ fixed)-Dual iter 1)，该问题无界，但是有一个极射线

$$\text{extreme ray} = \left[2,\ -1,\ -1,\ -2,\ 0,\ 0,\ -1,\ -1\right]^{\mathrm{T}}$$

也就是说，

$$\alpha_1 = 2,\quad \alpha_2 = -1,\quad \alpha_3 = -1$$

$$\beta_1 = -2,\quad \beta_2 = 0,\quad \beta_3 = 0,\quad \beta_4 = -1,\quad \beta_5 = -1$$

接下来我们要用这个极射线构造一个 Benders Feasibility Cut，也就是下面的数学表达式：

$$\left(\boldsymbol{\alpha}_r^i\right)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y}) \leqslant 0$$

但是这个式子是一个矩阵相乘的形式。其中，$\boldsymbol{\alpha}_r^i$ 就是我们刚刚得到的极射线。而 $(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})$ 就是 (Subproblem($\bar{\boldsymbol{y}}$ fixed)) 中的右端常数，同时也是其对偶问题 (Subproblem($\bar{\boldsymbol{y}}$ fixed)-Dual) 中的目标函数的系数，因为 (Subproblem($\bar{\boldsymbol{y}}$ fixed)-Dual) 的目标函数正是 $(\boldsymbol{b}-\boldsymbol{B}\bar{\boldsymbol{y}})^{\mathrm{T}}\boldsymbol{\alpha}$。首先得到（注意，由于要生成割平面，因此 $\boldsymbol{y}$ 变成了决策变量）

$$(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})^{\mathrm{T}} = \left[8,\ 3,\ 5,\ 8y_1,\ 3y_2,\ 5y_3,\ 5y_4,\ 3y_5\right]$$

然后将极射线的方向向量 $\boldsymbol{\alpha}_r^i$ 以及 $(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})$ 代入 Benders Feasibility Cut 的表达式 $\left(\boldsymbol{\alpha}_r^i\right)^{\mathrm{T}}(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y}) \leqslant 0$，得到

$$(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})^{\mathrm{T}} \cdot \boldsymbol{\alpha}_r^i = \left[8,\ 3,\ 5,\ 8y_1,\ 3y_2,\ 5y_3,\ 5y_4,\ 3y_5\right] \cdot \begin{bmatrix} 2 \\ -1 \\ -1 \\ -2 \\ 0 \\ 0 \\ -1 \\ -1 \end{bmatrix} \leqslant 0$$

因此，产生的新 Benders Feasibility Cut 就是

$$8 - 16y_1 - 5y_4 - 3y_5 \leqslant 0$$

17.2.3 第 2 次迭代

将上一次迭代产生的 Benders Feasibility Cut 添加到我们的松弛主问题 (Initial Master Problem Iter 1) 中，更新后的 (Initial Master Problem Iter 1) 变化为

$$
\begin{aligned}
\min \quad & 7y_1+7y_2+7y_3+7y_4+7y_5+q && \text{(Master Problem Iter 2)}\\
\text{s.t.} \quad & 16y_1+5y_4+3y_5 \geqslant 8 && (17.58)\\
& q \geqslant 0 && (17.59)\\
& y_1,\ y_2,\ y_3,\ y_4,\ y_5 \in \{0,1\} && (17.60)
\end{aligned}
$$

求解上述模型 (Master Problem Iter 2) 可以得到一个最优解

$$
\begin{aligned}
&\bar{\boldsymbol{y}} = \begin{bmatrix}1, & 0, & 0, & 0, & 0\end{bmatrix}^{\mathrm{T}}\\
&\bar{q} = 0
\end{aligned}
$$

注意，这一次迭代中，$\bar{\boldsymbol{y}}$ 的取值发生了变化。将新的固定的 $\bar{\boldsymbol{y}}$ 代入子问题 (Subproblem ($\bar{\boldsymbol{y}}$ fixed)-Dual)，更新子问题 (Subproblem($\bar{\boldsymbol{y}}$ fixed)-Dual) 为

$$
\begin{aligned}
\max \quad & 8\alpha_1+3\alpha_2+5\alpha_3+8\beta_1+0+0+0+0 && \text{(SP}(\bar{\boldsymbol{y}}\text{ fixed})\text{-Dual iter 2)}\\
\text{s.t.} \quad & \alpha_1+\beta_1 \leqslant 1 && (17.61)\\
& \alpha_2+\beta_2 \leqslant 1 && (17.62)\\
& \alpha_3+\beta_3 \leqslant 1 && (17.63)\\
& \alpha_1+\alpha_3+\beta_4 \leqslant 1 && (17.64)\\
& \alpha_1+\alpha_2+\beta_5 \leqslant 1 && (17.65)\\
& \alpha_1,\ \alpha_2,\ \alpha_3 \text{ free} && (17.66)\\
& \beta_1,\ \beta_2,\ \beta_3,\ \beta_4,\ \beta_5 \leqslant 0 && (17.67)
\end{aligned}
$$

求解上述更新后的子问题，依然是无界。但是仍然可以得到新的极射线

$$
\text{extreme ray} = \begin{bmatrix}0, & 0, & 1, & 0, & 0, & -1, & -1, & 0\end{bmatrix}^{\mathrm{T}}
$$

也就是说，

$$
\begin{aligned}
&\alpha_1 = 0, \quad \alpha_2 = 0, \quad \alpha_3 = 1\\
&\beta_1 = 0, \quad \beta_2 = 0, \quad \beta_3 = -1, \quad \beta_4 = -1, \quad \beta_5 = 0
\end{aligned}
$$

根据这个极射线，我们仍然可以构造一个 Benders Feasibility Cut，即

$$(\boldsymbol{b}-\boldsymbol{B}\boldsymbol{y})^{\mathrm{T}}\cdot\boldsymbol{\alpha}_r^i=\begin{bmatrix}8, & 3, & 5, & 8y_1, & 3y_2, & 5y_3, & 5y_4, & 3y_5\end{bmatrix}\cdot\begin{bmatrix}0\\0\\1\\0\\0\\-1\\-1\\0\end{bmatrix}\leqslant 0$$

整理得

$$5-5y_3-5y_4\leqslant 0$$

17.2.4 第 3 次迭代

接下来继续更新松弛主问题。加入上一次迭代产生的 Benders Feasibility Cut，将主问题 (Master Problem Iter 2) 更新为

$$\begin{aligned}
\min\quad & 7y_1+7y_2+7y_3+7y_4+7y_5+q && && && && && && && && \text{(Master Problem Iter 3)}\\
\text{s.t.}\quad & 16y_1 && && +5y_4+3y_5 && \geqslant 8 && && && && && && && &&